



Universidad Politécnica
de Madrid



**Escuela Técnica Superior de
Ingenieros Informáticos**

Grado en Ingeniería Informática

Trabajo Fin de Grado

**Desarrollo de una Herramienta Basada
en Recomendación de Libros en una
Tienda Online**

Autor: Sergio Gonzalo Vallinas

Tutora: Victoria Rodellar Biarge

Madrid, junio 2020

Este Trabajo Fin de Grado se ha depositado en la ETSI Informáticos de la Universidad Politécnica de Madrid para su defensa.

Trabajo Fin de Grado

Grado en Ingeniería Informática

Título: Desarrollo de una Herramienta Basada en Recomendación de Libros de una Tienda Online

Junio 2020

Autor: Sergio Gonzalo Vallinas

Tutor:

Victoria Rodellar Biarge

Departamento de Arquitectura y Tecnología de Sistemas Informáticos

ETSI Informáticos

Universidad Politécnica de Madrid

Agradecimientos

En primer lugar, me gustaría agradecer a mi tutora Victoria su dedicación y su ayuda, sin las cuales, este trabajo no habría salido adelante. Gracias por haberlo hecho siempre con una sonrisa.

A la empresa que ha hecho posible todo este proyecto, JOT Internet Media, José, Chesco y todo el departamento de tecnología. Nunca podré agradecer lo suficiente su confianza.

A mi familia, en primer lugar, a mis padres, mi hermana y mi abuela. Principales autores de lo que hoy en día he conseguido y testigos principales de todas mis frustraciones. Gracias por vuestra paciencia y confianza. También a mis tíos y primos, apoyo constante involuntario, siempre a mi lado para lo que he necesitado.

A Tita, por ser un punto de apoyo y desahogo siempre que lo he necesitado. Por estar siempre al pie del cañón para sacarme una sonrisa o darme ánimos cuando más lo he requerido.

A mis amigos, los que no necesitan moverse por interés, los que dan lo que sea por verte feliz. Los de siempre. Los Caudos. Gracias por haberme acompañado en la etapa más difícil de mi vida y siempre con una carcajada por delante sin sobrepasar la línea. Gracias por ser un pilar fundamental en mi vida. Sin que sientan celos, pero, sobre todo a mi fiel compañero Iván, que ha sido más que un apoyo en esta etapa, y que, sin exagerar, es el principal artífice de que hoy consiga este logro. Y por último, a Daniel y Desirée “to give without expecting anything in return”.

A mi compañera de viaje, mi mejor amiga, la persona que más me ayuda, la que me aconseja, la que me da todos los ánimos del mundo, la que me ayudó a sacar adelante estadística, la que me alegra la vida sonriendo. La que siempre está ahí. Gracias.

Se acaba como he dicho la etapa más difícil de mi vida, pero también gracias a ella veo la vida de otra manera. Sabiendo que, todo salvo la muerte tiene solución. Esto va por ellos, por los que lamentablemente hoy no están y que hicieron posible que hoy sea la persona que soy. Mis queridos y añorados abuelos: Aser, Tino y Carmen.

“Por cuidar lo curioso, no olvides lo necesario”

Resumen

Los sistemas de recomendación son herramientas que generan sugerencias sobre un objeto de estudio específico en base a las preferencias y opiniones dadas (o inferidas) por los usuarios.

Hoy en día estos sistemas están presentes en todas las plataformas que ofrecen contenido o plataformas dedicadas a la venta online. Estas herramientas se encuentran en plena ebullición, evolucionando día tras día y es que las empresas han encontrado un filón debido a los grandes beneficios que estos reportan. Si nos paramos a pensar en plataformas que ofrezcan contenido digital (series, películas, etc.) nos vienen a la cabeza varios ejemplos que son muy similares, pero por alguna razón, los usuarios usan más una que la otra. ¿Por qué? Esto tiene una explicación muy sencilla. Netflix es una de las empresas que más cuidan este apartado. ¿Qué le supone? Más de un 80% de las visualizaciones que realizan sus usuarios son recomendaciones que les ofrece el sistema. He aquí su punto diferencial.

El objetivo principal de este proyecto es aportar una herramienta basada en un sistema de recomendación de libros, la cual será capaz de hacer coincidir los productos disponibles en la tienda con los usuarios en función de la información que describe la oferta de libros y el perfil del usuario. Esto se creará con inteligencia artificial donde se aprenderá cuáles son los gustos del usuario.

El funcionamiento de esta aplicación consiste en utilizar distintas interacciones que realicen los lectores en nuestra herramienta para, más tarde, ser tratada y ofrecer las mejores recomendaciones. La información captura (píxel) se lleva a cabo con la plataforma Google Tag Manager. Una vez tenemos estos datos la herramienta es capaz de elaborar una lista con los libros que mayor porcentaje tienen de ser comprados en un futuro por nosotros. Para la realización la predicción automática se ha utilizado Python y su librería scikit-learn.

Palabras clave: Sistema de recomendación, aprendizaje automático, desarrollo software...

Abstract

Recommendation systems are tools that generate suggestions about a specific object of research based on the preferences and opinions given (or inferred) by users.

Today these systems are present in all platforms that offer content or platforms dedicated to online sales. These tools are bubbling, evolving every day. Companies have found a way to benefit from them. If we think about platforms that offer digital content (series, movies, etc.) several examples come to mind that are very similar, but for some reason, users use one more than the other. Why? This has a very simple explanation. Netflix is one of the companies that takes the most care of this section. What does it mean to them? More than 80% of the visualizations made by their users are recommendations offered by the system. This is its differential point.

The main objective of this project is to provide a tool based on a book recommendation system, which will be able to match the products available in the store with the users based on the information that describes the book offered and the user's profile. This will be created with artificial intelligence where the user's tastes will be learned.

The operation of this application consists in using different interactions that the readers make in our tool to, later, be treated and offer the best recommendations. The information captured (pixel) is carried out with the Google Tag Manager platform. Once we have this data, the tool is able to elaborate a list with the books that have the highest percentage of being bought in the future by us. Python and its library scikit-learn have been used for the realization of the automatic prediction tool.

Keywords: Recommendation system, machine learning, software development...

Índice de tablas

Tabla 4.1: Productos	15
Tabla 4.2: Usuarios	15
Tabla 4.3: Interacciones-usuario.....	16
Tabla 4.4: Usuario-audiencia	16
Tabla 4.5: Interés-audiencia	16
Tabla 4.6: Palabras clave extraídas	17
Tabla 4.7: Recomendaciones	17
Tabla 5.1: Coste del dominio	35
Tabla 5.2: Coste del material	36
Tabla 5.3: Coste de las licencias	36
Tabla 5.4: Coste del desarrollador	38

Índice de figuras

Figura 2.1: Taxonomía de los sistemas de recomendación.....	3
Figura 2.2: Matriz de factorización	4
Figura 2.3: Ejemplo de sistema basado en vecinos	5
Figura 3.1: Logo de Python	8
Figura 3.2: Logo de Javascript	9
Figura 3.3: Logo de PHP	10
Figura 4.1: Estructura del sistema de recomendación.....	13
Figura 4.2: Esquema para modificar la plantilla de Wordpress.....	19
Figura 4.3: Modificaciones al código.....	19
Figura 4.4: Código necesario en la hoja de estilos	20
Figura 4.5: Logo de Solid Inrup	21
Figura 4.6: Uso de librería solid-auth	21
Figura 4.7: Captura de información	21
Figura 4.8: Formulario de bienvenida	22
Figura 4.9: Escritura del Pod	23
Figura 4.10: Escritura en base de datos	23
Figura 4.11: Conexión con SQL Server	24
Figura 4.12: Borrado del Pod	24
Figura 4.13: Esquema del sistema	25
Figura 4.14: Panel principal del GTM	26
Figura 4.15: Panel de una etiqueta	27
Figura 4.16: Panel de un activador	27
Figura 4.17: Datos de configuración de Firebase	28
Figura 4.18: Descarga del blob	28
Figura 4.19: librerías de scikit-learn	29

Figura 4.20: Carga del dataset	29
Figura 4.21: Cruzado de dataframes	30
Figura 4.22: Creación de la matriz de puntuaciones	30
Figura 4.23: Calculo de las predicciones	30
Figura 4.24: Función para el cálculo basado en artículo	31
Figura 4.25: Función de búsqueda de perfiles similares	31
Figura 4.26: Recogida de datos	31
Figura 4.27: Comparativa de interacciones	32
Figura 4.28: Cálculo final	32
Figura 4.29: Esquema de base de datos	33
Figura 4.30: Tabla de audiencias	34
Figura 5.1: Lista de tareas realizadas	37
Figura 6.1: Acceso a Solomon	39
Figura 6.2: Login Solid	40
Figura 6.3: Registro del Pod	41
Figura 6.4: Panel del Pod	42
Figura 6.5: Formulario de registro	43
Figura 6.6: Panel de los datos insertados	44
Figura 6.7: Página principal de la tienda	45
Figura 6.8: Página principal de un artículo	46
Figura 6.9: Menú de un carrito	47
Figura 6.10: Recomendaciones mostradas	48

Tabla de contenidos

1	Introducción.....	1
1.1	Objetivos	1
1.2	Punto de partida	2
2	Estado del arte	3
2.1	Filtro colaborativo	4
2.1.1	Basado en vecinos cercanos	4
2.1.1.1	Basado en usuario	5
2.1.1.2	Basado en artículo	6
2.2	Basado en contenido	6
2.3	Modelo híbrido	7
3	Tecnologías usadas	8
3.1	Python.....	8
3.1.1	Pandas.....	8
3.2	Javascript	9
3.3	Php	10
3.4	SQL Server	10
3.5	Firebase	11
3.6	Google Tag Manager	11
4	Desarrollo	12
4.1	Base de datos	12
4.1.1	Diagrama	12
4.1.2	Tablas	14
4.1.2.1	Products	15
4.1.2.2	Users	15
4.1.2.3	User interactions	16
4.1.2.4	User audience	16
4.1.2.5	Interest audience	16
4.1.2.6	Keywords_extracted & Keywords_extracted_bulk	17
4.1.2.7	Recommendations	17
4.2	Página Web –Wordpress	18
4.2.1	Plantillas y pluggins	18
4.2.2	Child theme	18
4.3	Solid Pods	20
4.3.1	Carga del perfil.....	21
4.3.2	Escritura del pod	23
4.3.3	Borrado del pod	24
4.4	Sistema de recomendación	25

4.4.1	Diseño	25
4.4.2	Flujo del sistema	26
4.4.2.1	¿Cómo funciona Google Tag Manager?	26
4.4.2.2	Descarga de Firebase	28
4.4.2.3	Script recomendador	29
4.4.2.4	Script recomendador Firebase	32
4.4.2.5	Script recomendador por audiencias	33
5	Costes	35
5.1	Costes de infraestructura	35
5.1.1	Dominio	35
5.1.2	Material	35
5.2	Costes de desarrollo	36
5.2.1	Licencias software	36
5.2.2	Desarrollador	37
5.3	Coste final	38
6	Resultados	39
6.1	Acceso	39
6.2	Registro	40
6.3	Pod	42
6.4	Tienda	44
6.5	Recomendaciones	45
6.5.1	Ejemplo de uso	45
7	Conclusiones	49
8	Bibliografía	50
9	Anexo	51

1 Introducción

Los sistemas de recomendación se han ido incorporando poco a poco a nuestra vida hasta llegar a ser imprescindible para casi todas las plataformas online que visitamos cada día. Tal es su uso que pueden llegar a pasar desapercibidos, pero están ahí. Cuando abrimos nuestra aplicación de música siempre aparece un apartado que nos empieza a bombardear con distintas listas de canciones muy parecidas o del mismo estilo que las que hemos estado escuchado el día anterior. Un rato más tarde, desbloqueamos nuestro dispositivo electrónico para revisar en nuestra tienda on-line favorita si ha bajado el precio de la camiseta que vimos ayer, accedemos y... sorpresa, ahí está. La página web nos muestra directamente tanto los productos que estuvimos revisando anoche, como una colección de prendas de ropa que se ajustan mucho a nuestros gustos. Podría ser casualidad, pero no lo es.

En los siguientes apartados se va a describir la herramienta principal sobre la que se basa el proyecto, los sistemas de recomendación. Estos mecanismos se encargan de sugerir a los usuarios una selección de todos los productos que se ofrecen, siendo estos los que mejor se ajustan a las preferencias de cada consumidor en base a una serie de criterios que previamente han sido analizados.

Entendemos por sistema de recomendación aquel que compara el perfil del usuario con algunas características de referencia de los temas, y busca predecir el baremo o ponderación que el usuario le daría a un ítem que aún el sistema no ha considerado. Estas características pueden basarse en la relación o acercamiento del usuario con el tema o en el ambiente social del mismo usuario [1].

La finalidad de la recopilación de estos datos es conseguir ofrecer a los usuarios una recomendación personalizada de los productos, aumentando las probabilidades de una futura compra.

1.1 Objetivos

El principal objetivo de este proyecto es la elaboración de un sistema de recomendación que se encapsule en una tienda online de libros donde, a partir del perfil y actividad de los usuarios, utilizando técnicas de inteligencia artificial, se muestren unas determinadas recomendaciones al usuario.

Para el desarrollo de este trabajo se han establecido los siguientes objetivos a alcanzar:

- Creación de la tienda online en el conocido sistema de gestión de contenidos, Wordpress. La elección de crear la página web en este sitio es por petición expresa del cliente.
- Diseño de Bases de Datos relacionales.
- Diseño del módulo del trackeo (seguimiento) y medición de los usuarios con las herramientas de Google Tag Manager y Firebase. Estas herramientas permiten realizar el seguimiento y la medición de cualquier hito que acontezca en nuestro sitio.

- Diseño del sistema de recomendación de libros.

1.2 Punto de partida

El punto de partida de este trabajo fin de grado ha sido facilitado y desarrollado en conjunto gracias a la empresa donde empecé a realizar mis practicas curriculares, JOT Internet Media [2].

José Cabanilas, jefe del departamento de tecnología de la empresa, es mi tutor de empresa y junto con Iván García, ingeniero informático de este departamento, han sido los encargados de resolver las dudas que me han ido surgiendo a lo largo del desarrollo del trabajo.

Fueron ellos los que me ofrecieron participar en el desarrollo de un proyecto que tenía la empresa. Me explicaron cuáles eran los objetivos y en que iba a consistir dicho trabajo. Además, me comentaron cuales eran las herramientas que iba a tener que usar y de que material, propio de la empresa, disponía.

JOT Internet Media, es una empresa dedicada al marketing digital mediante la generación de tráfico eficiente y de calidad, con un crecimiento grande y rápido, haciendo que se posicione como una de las principales empresas en marketing digital.

Entre sus actividades, está la de ayudar a sus socios a la adquisición de nuevos públicos relevantes para maximizar los beneficios de sus negocios. Este aumento de su tráfico web se traduce en un servicio de monetización para aumentar los ingresos por usuario y generar ingresos incrementales. Además, la inmensa cantidad de datos que generan con los proyectos pueden resultar relevantes para otras empresas y de los cuales he podido disponer para la realización del sistema de recomendación.

2 Estado del arte

A continuación, se va a describir la herramienta principal sobre la que se basa el proyecto, los sistemas de recomendación. Este mecanismo se encarga de sugerir a los usuarios una selección de todos los productos que se ofrecen, estos serán aquellos que mejor se ajustan a las preferencias de cada consumidor en base a una serie de criterios que previamente han sido analizados.

La finalidad de la recopilación de estos datos es conseguir ofrecer a los usuarios una recomendación personalizada de los productos, aumentando en casi un tercio las posibilidades de una futura compra.

Como ya hemos mencionado anteriormente, un sistema de recomendación tendrá como objetivo principal adelantarse y acercar a los usuarios los diferentes tipos de opciones que ofrece el sitio web para que tomen una decisión.

En la actualidad se han desarrollado miles de algoritmos, formas y métodos para incorporar a estos sistemas. A continuación, vamos a ver como se clasifican los más destacados.

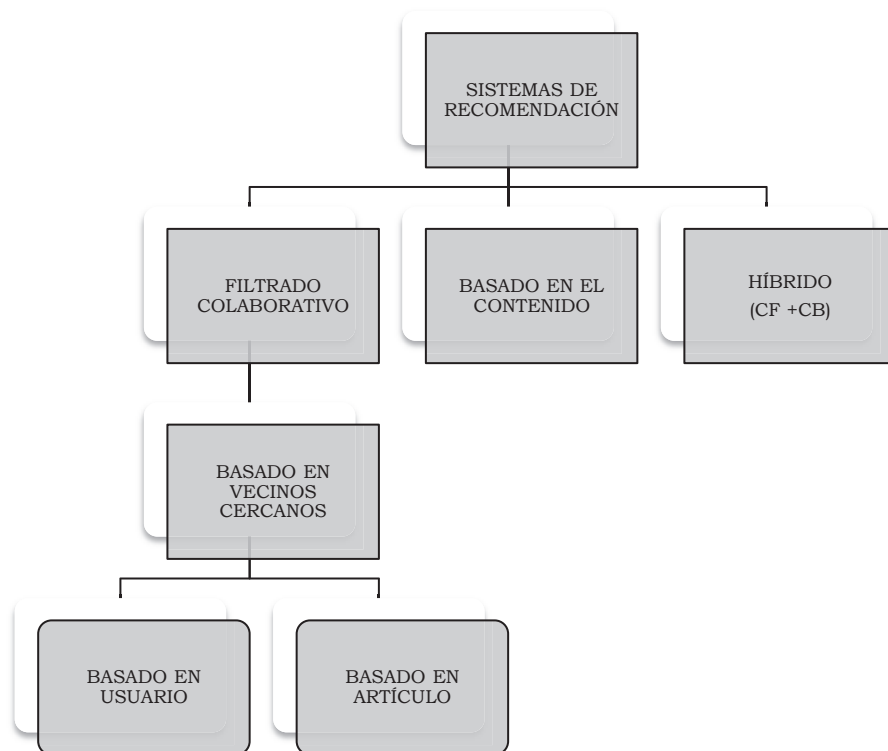


Figura 2.1: Taxonomía de los sistemas de recomendación. .

En los siguientes apartados se describirán uno a uno los elementos que componen la taxonomía.

2.1 Filtro colaborativo

El filtro colaborativo [3] es una técnica utilizada para hacer predicciones automáticas acerca de los intereses de un usuario mediante la recopilación de preferencias o información sobre sus gustos. Una explicación general sería que, si una persona, por ejemplo, X, tiene la misma opinión que Y sobre algún tema, es más probable que X tenga igual opinión de Y respecto a otro tema diferente que la de Z, siendo esta última elegida aleatoriamente. Por tanto, este método no necesita nada más que un histórico de preferencias de los usuarios sobre un conjunto de artículos. Se centra exclusivamente en las interacciones usuario-artículo.

2.1.1 Basado en vecinos cercanos

Este otro método, perteneciente a la rama del filtrado colaborativo, está basado en el “vecindario” del usuario investigado, es decir, estudiando distintos patrones se pueden averiguar a partir de los gustos de los usuarios las similitudes de distintos elementos [4]. El objetivo principal es utilizar las puntuaciones dadas por estos vecinos para poder predecir cuales serían las calificaciones del usuario.

¿Pero, que se entiende por vecino? Entendemos por vecino de un usuario a los distintos individuos que se colocarán en una matriz de $n \times m$, siendo n el consumidor (user) y m el artículo (item). El proceso consiste averiguar mediante distintos cálculos las similitudes entre el objetivo y todos los demás, seleccionar los usuarios similares y tomar el promedio ponderado de las clasificaciones de estos usuarios.

	item1	item2	item3	item4	item5	item6
user1						
user2						
user3						
user4						

Figura 2.2: Matriz de factorización. Fuente: <https://dzone.com/articles/recommendation-engine-models>

A continuación, se muestra un ejemplo gráfico de la idea principal de esta técnica. Por un lado, tenemos a Juan, que ha entrado en nuestra página web y ha visitado una serie de libros que luego ha terminado comprando. Y, por otro lado, tenemos a María que hace unos días compro los mismos libros que Juan y ahora está buscando otro libro para leerse estos días. Nuestro sistema detecta que Juan y María tienen o pueden tener gustos similares por su actividad pasada, por tanto, recomendamos a María un libro que Juan había comprado previamente.

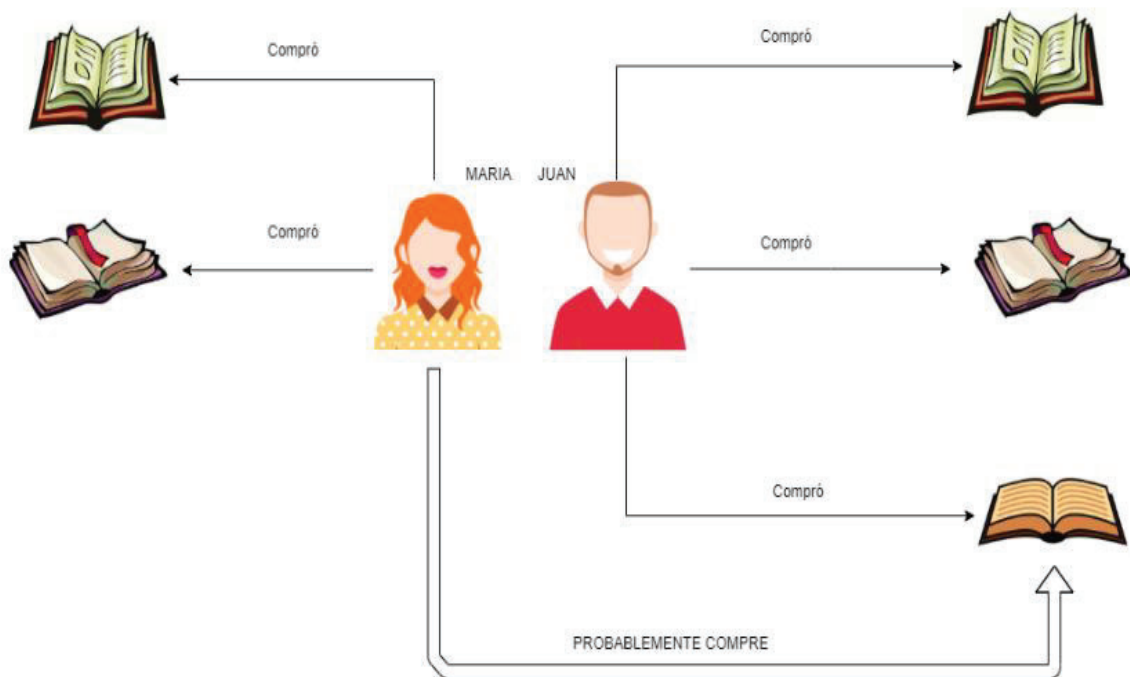


Figura 2.3 Ejemplo de sistema basado en vecinos.

2.1.1.1 Basado en usuario

Este otro método se basa en encontrar un conjunto de usuarios que son similares o tienen intereses parecidos a los del usuario A y predice la calificación que este usuario podría dar al producto. Sobre la base de esas calificaciones, el sistema recomienda al usuario A los artículos revisados o calificados por los usuarios de ese conjunto.

Para calcular el usuario más cercano de otro se pueden utilizar diferentes funciones de similitud, las dos más importantes son: la distancia de Jaccard y la distribución de Pearson [5].

El coeficiente de correlación de Pearson, pensado para variables cuantitativas, es un índice que mide el grado de covariación entre distintas variables relacionadas linealmente. La expresión matemática se indica a continuación:

$$\rho_{X,Y} = \frac{\sigma_{XY}}{\sigma_X \sigma_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y} \quad (1)$$

Donde X e Y son variables aleatorias sobre un conjunto aleatorio, σ_{XY} es la covarianza de (X, Y), σ_X es la desviación estándar de X y σ_Y de Y.

El coeficiente de Jaccard mide el grado de similitud entre dos conjuntos, sea cual sea el tipo de elementos [6]. Este algoritmo desarrollado por Paul Jaccard, compara la similitud y la diversidad de la muestra conjuntamente. El coeficiente de Jaccard mide la similitud entre conjuntos de muestras finitas, y se define como el tamaño de la intersección dividido por el tamaño de la unión de los conjuntos. Seguidamente se indica la expresión:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|} \quad (2)$$

Donde: A y B son conjuntos de muestras.

La distancia Jaccard, es complementaria al coeficiente de Jaccard y se obtiene restando uno menos el coeficiente de Jaccard, o, de manera equivalente, dividiendo la diferencia de los tamaños de la unión y la intersección de dos conjuntos del tamaño de la unión, como puede verse a continuación:

$$d_J(A, B) = 1 - J(A, B) = \frac{|A \cup B| - |A \cap B|}{|A \cup B|} \quad (3)$$

Donde: A y B son conjuntos de muestras.

2.1.1.2 Basado en artículo

Este enfoque es como el anterior, pero en lugar de encontrar un conjunto de usuarios, encuentra un conjunto de elementos similares que un usuario ha calificado positivamente. Sobre esa base de calificaciones, el sistema predecirá la puntuación del artículo que aún no ha sido calificado por el usuario que estamos investigando. Transponiendo la matriz mencionada anteriormente calcularíamos la similitud entre artículos. Así mismo para calcular la similitud entre dos artículos podemos utilizar las mismas funciones que en método anterior.

2.2 Basado en contenido

Los sistemas de recomendación que utilizan este tipo de método [7] tratan de recomendar artículos como los que le han gustado a un determinado usuario en el pasado. Por tanto, el proceso que realiza consiste en hacer coincidir los atributos de un perfil de usuario donde se almacenan las preferencias e intereses con los atributos de un objeto, con el objetivo de recomendar al usuario nuevos artículos que podrían serle interesantes. Para este tipo de filtrado se requiere una gran cantidad de información acerca de las características de los artículos, en vez de utilizar las interacciones y retroalimentaciones de los usuarios como pudieran ser los ejemplos

anteriormente vistos, por ejemplo, en nuestro caso sería de utilidad la categoría de un libro, su autor, su editorial, etc. Por tanto, este tipo de mecanismos son los más adecuados para situaciones en los que se precisa de más información acerca de los productos que de los consumidores.

La idea principal aquí es que los usuarios que han estado de acuerdo en el pasado tienden a estarlo también en el futuro. Solo es necesario la preferencia histórica.

2.3 Modelo híbrido

Y por último tenemos el sistema de recomendación híbrido, denominado así porque combina las distintas estrategias mencionadas anteriormente. El objetivo de este modelo es aprovechar las ventajas que otorgan cada tipo de sistema, por este motivo un sistema de recomendación híbrido ofrece mejor rendimiento al ser más precisa la recomendación. Un sistema pasa a denominarse híbrido en el momento que combina dos estrategias.

Este mecanismo será el elegido para integrar la parte del sistema recomendador.

3 Tecnologías usadas

A continuación, se presentan los lenguajes de programación utilizados para el proyecto, modelos de bases de datos y tecnologías asociadas.

3.1 Python

Python fue creado a finales de los ochenta por Guido van Rossum en el Centro para las Matemáticas y la Informática en los Países Bajos, como sucesor del lenguaje de programación ABC.

Una de las características más importantes y que ha sido atractivo para millones de programadores en todo el mundo es que es un lenguaje de programación multiparadigma, es decir, ofrece herramientas para trabajar varios estilos de programación como programación orientada a objetos, imperativa, funcional, etc. Estos son solo algunos ejemplos, ya que mediante el uso de extensiones se podrían usar más estilos. Con Python se pueden implementar distintos tipos de aplicaciones como scripts, aplicaciones web y aplicaciones de escritorio.

Otra característica que posee este lenguaje y que le ha permitido ganar muchísima popularidad es la gran cantidad de librerías de las que dispone. Entendemos por librería o biblioteca al conjunto de implementaciones funcionales codificadas en un lenguaje de programación que ofrece una interfaz bien definida para la funcionalidad que se invoca. Su finalidad es ser utilizada por otros programas, independientes y de forma simultánea consiguiendo realizar tareas sin tener que desarrollar completamente el método. Con una llamada a la librería estaríamos ahorrando todo ese proceso. Con estas herramientas podremos realizar visualizaciones, cálculos numéricos, podremos tratar análisis de datos, realizar aprendizajes automáticos, etc. [8]



Figura 3.1 Logo de Python

3.1.1 Pandas

Entre las librerías que usaremos para el desarrollo del proyecto hay una que ha tomado mayor protagonismo y por tanto se ha convertido en la más importante y por ello se menciona. Esta es la librería Pandas. [9]

Pandas fue desarrollada por Wes McKinney en el año 2008 mientras trabajaba en AQR Capital. Es una de las librerías de Python más útiles para los científicos que manejan gran cantidad de datos. Podemos distinguir dos tipos de estructuras:

- Series: para datos en una dimensión.
- DataFrame: para datos en dos dimensiones.

Estas son las estructuras de datos más usadas en campos como las finanzas, las estadísticas, ciencias sociales y muchas áreas de ingeniería. Pandas destaca por lo fácil y flexible que hace la manipulación de datos y el análisis de datos.

3.2 Javascript

Javascript es un lenguaje de programación orientado a objetos. Nos permite realizar actividades en páginas web. Su principal característica es que no necesita ser compilado ya que son los navegadores los que se encargan de asimilar el código leído y ejecutarlo.

En 1995, Brendan Eich creó Mocha, que más tarde se convertiría en el lenguaje para programar en web más importante que ha existido. En un principio se diseñó una sintaxis similar a C, aunque adopta nombres y convenciones del lenguaje de programación Java. Sin embargo, Java [10] y JavaScript [11] son totalmente diferentes.

Actualmente es utilizado para enviar y recibir información del lado del servidor gracias a tecnologías como AJAX. JavaScript se interpreta en el usuario al mismo tiempo que las sentencias van descargándose junto con el código HTML consiguiendo que en una página web haya mayor interactividad con los usuarios y mejore su experiencia.



Figura 3.2. Logo de JavaScript

3.3 Php

Php es otro de los lenguajes de programación en los que se ha desarrollado este proyecto. Es un lenguaje de código abierto creado por Rasmus Lerdorf en 1995 [12]. Muy popular para el desarrollo de páginas web ya que puede ser incrustado en el lenguaje de marcas, HTML. Php se sitúa del lado del servidor diseñado para el preprocesado de texto plano en UTF-8 [13].

Algunas de las características más importantes podrían ser:

- Está orientado al desarrollo de aplicaciones web dinámicas.
- Fácil de aprender.
- Capacidad de conexión con la mayoría de los motores de base de datos
- De código libre.
- No requiere definición de tipos de variables.
- Podemos expandir su potencial a través de módulos.



Figura 3.3. Logo de Php

3.4 SQL Server

Microsoft SQL Server es un sistema de gestión de base de datos relacional desarrollado como un producto de software con la función principal de almacenar y recuperar datos, desarrollado por la empresa Microsoft. Fue lanzado el 24 de abril de 1989 [14]. Los servidores SQL Server suelen presentar como principal característica una alta disponibilidad al permitir un gran tiempo de actividad y una conmutación más rápida.

Algunas características de este sistema son [15]:

- Soporte de transacciones.
- Escalabilidad, estabilidad y seguridad.
- Soporte de procedimientos almacenados.
- Incluye también un potente entorno gráfico de administración

- Uso de comandos DDL y DML gráficamente.
- Permite trabajar en modo cliente-servidor, donde la información y datos se alojan en el servidor y las terminales o clientes de la red solo acceden a la información.
- Permite administrar información de otros servidores de datos.

3.5 Firebase

Se trata de una plataforma para el desarrollo de aplicaciones web y aplicaciones móviles desarrollada por Google en 2014 [16]. Se ubica en la nube, compuesto por Google Cloud Platform. Posee un conjunto de utilidades para implementar y garantizar aplicaciones de alta calidad, proporcionadas por Google, y herramientas para la sincronización de proyectos.

Gracias a esta aplicación podremos gestionar de manera sencilla todas las interacciones que realicen los usuarios en nuestro espacio web, ya que, ofrece diferentes servicios como base de datos, autenticación, nube de almacenamiento, hosting, test lab y crashreporting [17].

3.6 Google Tag Manager

Google Tag Manager es una herramienta que administra la gestión de etiquetas o fragmentos de código con el objetivo de poder hacer un seguimiento y medición de cualquier actividad que ocurra en nuestro sitio web de forma fácil y rápida. Fue lanzado en octubre de 2012 y su objetivo principal era ayudar a empresas dedicadas al marketing digital en las tareas de gestión y seguimiento de campañas [18].

Dentro de la plataforma podemos hacer pequeñas inserciones de código en Javascript que más tarde será añadido a nuestra página web. Estos códigos que hemos añadido son herramientas de análisis de tráfico como Google Analytics, campañas de marketing como Google Adwords/Facebook Ads, y herramientas de optimización como Google Optimize/Hotjar/CrazyEgg.

4 Desarrollo

En el siguiente documento se describe el desarrollo que se ha llevado a cabo para implementar el proyecto del sistema de recomendación y cómo funcionará. Además, durante el desarrollo de la documentación iremos relatando cuales han sido los pasos que hemos seguido para construir las bases del proyecto.

4.1 Base de datos

En esta sección se explica la arquitectura global de la base de datos implementada, cómo funcionará y cuáles son los componentes básicos y sus conexiones.

Para poner en práctica un sistema de recomendaciones es necesario implementar fuentes de datos que permitan la correlación entre ellas. En nuestro caso, este sistema de recomendación está formado por tres componentes diferentes: usuarios, productos y las audiencias. En el caso de las audiencias, JOT, como experto en marketing, es el responsable de elegir qué información de los usuarios definirá las características de las audiencias.

4.1.1 Diagrama

En el diagrama siguiente se muestra la arquitectura global que tendrá el servicio de recomendación.

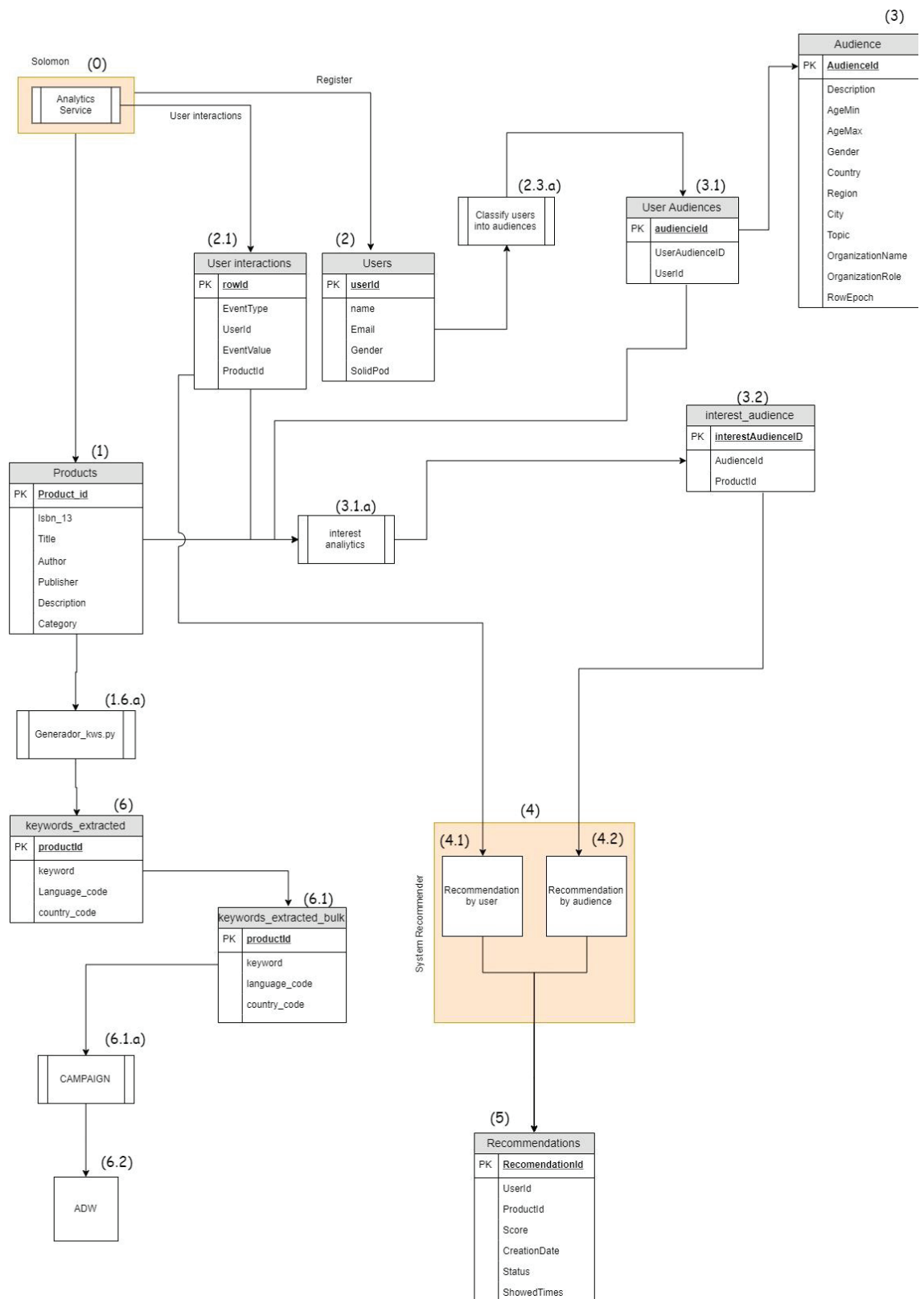


Figura 4.1 Estructura del sistema de recomendación.

Se puede observar el flujo de datos, los servicios de procesamiento de datos y los componentes de almacenamiento de datos. En el siguiente párrafo se describe detalladamente esta arquitectura, haciendo referencia a los componentes según los números indicados en la figura:

- En primer lugar, el sitio web. Se decidió llamarlo Solomon, (0) y su servicio de análisis nos proporciona todos los datos para el servicio de recomendación.

- Necesitamos dividir en grupos a todos los usuarios (2). El proceso, 2.3.a, se encarga de clasificar a los usuarios en función de algunas de sus características. De esta manera, surge una tabla llamada Audiencias (3) con la tabla de Audiencias de Usuarios (3.1).

- Una vez clasificados los usuarios en audiencias, se produce un proceso de análisis de interés (3.1.a) con un complejo algoritmo que se encarga de analizar qué productos son potencialmente interesantes para cada público. Lógicamente, este proceso requiere de hasta tres fuentes de datos diferentes:

1. Las audiencias
2. Los productos
3. Las interacciones de los usuarios que pertenecen al público correspondiente.

Cuando este proceso finaliza, se dejan los resultados en la tabla de Audiencias de interés (3.2), que representa los productos que interesan a cada audiencia.

Se dejarán los resultados en una nueva tabla de la base de datos (5), que será el resultado final de nuestro sistema de recomendaciones. De esta manera, será posible mostrar a cada usuario los productos que tienen la mayor calidad y exactitud de recomendación.

Por otro lado, utilizando un sencillo script de Python (1.6.a) conseguiremos extraer una serie de palabras clave a partir de los títulos de los productos y sus descripciones.

Con estos datos nutriremos la tabla “Keywords_extracted” (6) y que utilizaremos para subir campañas (6.1.a) a Google Ads (6.2), esto es una plataforma de la empresa Google que se utiliza para ofrecer publicidad patrocinada a potenciales anunciantes. Más adelante, se explicará con detalle el objetivo de este proceso. Todo este proceso estará ajeno al TFG, pero se desarrollará para la empresa.

4.1.2 Tablas

A continuación, se describen las tablas que han sido usadas para la implementación de la base de datos de Solomon, tal y como aparece en el diagrama mostrado anteriormente. En el caso de la tabla “Recomendaciones” explicaremos los atributos de la tabla porque no son tan descriptivos como el resto.

4.1.2.1 Products

La siguiente tabla contendrá toda la información de los productos, en nuestro caso, libros. Los detalles de cada ejemplar se ingresarán en el servicio web donde se mostrarán a los posibles consumidores.

Products	
PK	<u>Product_id</u>
	Isbn_10
	Isbn_13
	Title
	Author
	Publisher
	PublishedDate
	Description
	Category
	AverageRating_Google
	RatingCount_Google
	MaturityRating_Google
	Language
	PageCount
	Edition
	Sagald

Tabla 4.1 Productos

4.1.2.2 Users

Se almacenará toda la información que define a los usuarios (principalmente datos demográficos y personales). Esta información será proporcionada por el sitio web que se llevará a cabo en el registro.

Users	
PK	<u>userId</u>
	Name
	Surname
	Email
	Gender
	Birthdate
	Country
	Region
	city
	Postal Code
	Address
	Language
	TopicInterest
	SolidPod

Tabla 4.2 Usuarios

4.1.2.3 User interactions

El propósito de esta tabla es almacenar todas las interacciones más significativas de los usuarios. Esta información se obtendrá del servicio de análisis web del proyecto, con herramientas como Google Analytics, Google Tag Manager, etc.

User interactions	
PK	rowId
	EventType
	UserId
	EventValue
	ProductId

Tabla 4.3 Interacciones-usuario

4.1.2.4 User audience

Tabla que relaciona cada usuario y su público. Simplemente contendrá las identificaciones de la audiencia y las identificaciones de los usuarios con una breve descripción opcional.

User Audiences	
PK	audiencelId
	UserAudienceID
	UserId

Tabla 4.4 Usuario-audiencia

4.1.2.5 Interest audience

Como la anterior, es una tabla de relación. En este caso, se encarga de almacenar los productos que interesan a cada público.

interest_audience	
PK	interestAudienceID
	AudiencelId
	ProductId

Tabla 4.5 Interés-audiencia

4.1.2.6 Keywords_extracted & Keywords_extracted_bulk

En estas tablas se almacenarán las palabras clave que se seleccionarán de las distintas características de los productos, como pudiera ser el título o su descripción. Gracias a esto podremos crear campañas para subirlas a Adwords.

keywords_extracted	
PK	productId
	keyword
	Language_code
	country_code

Tabla 4.6 Palabras clave extraídas

4.1.2.7 Recommendations

Todos los resultados del motor de recomendación estarán aquí.

- RecommendationId: ID de la recomendación.
- UserId: ID del usuario.
- ProductId: ID del producto.
- Score: puntuación de la recomendación calculada previamente.
- CreationDate: cuando se registró la recomendación.
- Status: para indicar que una recomendación es válida o no. Una recomendación no es válida, por ejemplo, cuando el usuario la ha rechazado o cuando la recomendación es demasiado antigua o cuando el usuario compra el libro recomendado.

Recommendations	
PK	RecommendationId
	UserId
	ProductId
	Score
	CreationDate
	Status
	ShowedTimes

Tabla 4.7 Recomendaciones

4.2 Página Web –Wordpress

WordPress es un sistema de gestión de contenidos creado el 27 de mayo de 2003 por Matt Mullenweg y Mike Little, enfocado a la creación de cualquier tipo de página web [19].

La empresa decidió que este sería el recurso a utilizar para diseñar la página web. En un primer momento el desarrollo se realiza de forma local, pero con vistas a un alojamiento en un servidor propio. Algunos de los puntos fuertes de Wordpress y que ha hecho que la compañía JOT Internet Media se decante por este servicio es sobre todo por contar con poseer la peculiaridad de ser un proyecto de software libre y de fuentes abiertas. Esto quiere decir que su código es creado y mantenido por una comunidad de miles de desarrolladores voluntarios, lo que garantiza su continuidad.

A pesar de ser una plataforma que nos da la opción de crear webs y sus contenidos de forma visual, sin tener que programar, también nos da posibilidad de poder modificar partes de código de una forma peculiar, “enganchándonos” a él. Esta ha sido la forma de implementar la página web de Solomon y ajustarla a nuestros deseos y condiciones debido a que al funcionar como un “repositorio público”, no se recomienda modificar directamente el código, aunque sea de forma local. Esto es debido a que en el caso de que se lance alguna actualización en la plataforma sobrescribiría nuestros cambios, eliminando nuestros avances.

4.2.1 Plantillas y pluggins

Las plantillas o temas y los plugins de WordPress tienen un concepto muy similar. La primera funciona a modo de diseños ya fabricados, con su estilo de maquetación, sus tipos de letra, tamaños de letra, su conjunto y combinaciones de colores, etc.

Los plugins, en cambio, son un componente adicional con una funcionalidad muy concreta. Se añade de forma sencilla a una aplicación existente para ampliarla con una nueva funcionalidad. Es decir, es código ya montado que se “engancha” a nuestro código principal para poder funcionar.

4.2.2 Child theme

Como hemos mencionado anteriormente, el método más profesional para el desarrollo y modificación de cualquier elemento que se relacione con Wordpress es el método de “ganchos”.

Ocurre igual si queremos modificar un “theme”, esto se hace con el conocido, “child-theme”. Su uso consiste en evitar perder los cambios hechos en tu “theme” cuando se actualice. Los “child-theme” utilizan los archivos e información de un “tema padre”, es decir, del tema principal. Este es un tema que hereda las funcionalidades básicas de un tema principal o tema padre y permite mejorarlas, modificarlas y ampliarlas. Por ejemplo, agregar miniaturas a publicaciones, agregar tamaños adicionales a las miniaturas, añadir widgets, cambiar el texto que aparece después del extracto de un post, añadir estilos nuevos, etc.

Nuestro “child-theme” no se actualiza, sólo hereda las funcionalidades del tema padre. De tal forma, Wordpress siempre comprobará si existe algún “child” y en ese caso, este será lo primero que ejecute.

Para poder empezar a modificar la plantilla que nos proporcionaba Wordpress, debíamos crear primero una carpeta que tuviese el mismo nombre que el tema elegido añadiendo “-child”. Una vez creado debemos crear dos archivos principales que se van a convertir en la base de todos los cambios que queramos hacer, estos son **functions.php** y **style.css**.

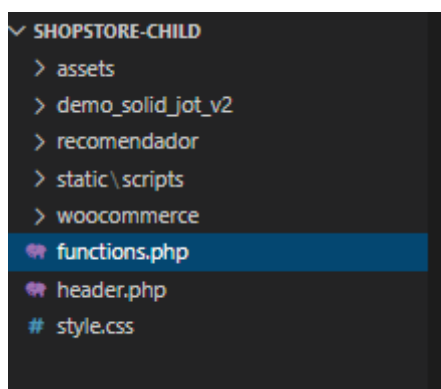


Figura 4.2 Esquema para modificar la plantilla de Wordpress.

En el primer archivo se añade el siguiente fragmento de código:

```
function my_theme_enqueue_styles() {  
  
    $parent_style = 'parent-style'; // Estos son los estilos del tema padre recogidos por el tema hijo.  
  
    wp_enqueue_style( $parent_style, get_template_directory_uri() . '/style.css' );  
    wp_enqueue_style( 'child-style',  
        get_stylesheet_directory_uri() . '/style.css',  
        array( $parent_style ),  
        wp_get_theme()->get('Version')  
    );  
}  
add_action( 'wp_enqueue_scripts', 'my_theme_enqueue_styles' );
```

Figura 4.3 Modificaciones al código

En la función que se muestra arriba, simplemente lo que hacemos es asegurarnos de que a la hora de la ejecución de código por parte del navegador lo primero que haga antes de leer el tema padre de Wordpress sea ver el tema hijo que vamos a implementar. A partir de aquí tenemos vía libre para engancharnos al código del tema y hacer las modificaciones que creemos necesarias.

En el segundo archivo, **style.css**, se debe añadir la siguiente información en forma de comentario:

```

# style.css > ...
1  ✓ /*!
2    Theme Name: shopstore child
3    Theme URI: https://athemeart.com/downloads/shopstore-child/
4    Author: aThemeArt
5    Author URI: http://athemeart.com/
6    Description: ShopStore is a sleek multipurpose WordPress WooCommerce
7    Template:shopstore
8    Version: 4.1.3
9    License: GPLv3
10   License URI: http://www.gnu.org/licenses/gpl-3.0.html
11
12   Tags: one-column, two-columns, right-sidebar, left-sidebar, custom-he
13   */
14

```

Figura 4.4 Código necesario en la hoja de estilos

En este archivo podremos añadir los diferentes estilos que queramos dar a los objetos de nuestra nueva página. Gracias a la acción realizada en ***functions.php*** conseguiremos que se ejecuten todos estos estilos antes de que lo hagan los del tema padre. Esta es la manera que nos permite este servicio si queremos que nuestro tema herede algunas funcionalidades básicas para mejorarlas, modificarlas o ampliarlas.

4.3 Solid Pods

Para el almacenamiento de los datos de los usuarios se utiliza el servicio conocido como Solid Pods [20]. Solid es un acrónimo inglés de Social Linked Data. Se trata de un proyecto de descentralización de datos en la web, el objetivo es conseguir que sea el usuario quien decide dónde almacenar sus datos, de esta forma se consigue mejorar la privacidad.

Una de las ventajas que tiene el proyecto y por las que ha sido elegido, además del de aumentar la privacidad, es el de dar una mayor comodidad al usuario, ya que posee la capacidad de compartir datos y el intercambio de información.

Para hacer uso de las aplicaciones que implementan Solid es necesario tener un pod y un WebID. Esta última entidad que hemos comentado, WebID, sirve para identificarse en Solid y acceder a los pods, así como conectar con otra gente que utiliza Solid.

A la hora de gestionar nuestra información, Solid ofrece varias alternativas. Una de ellas es hacerlo de forma personal, siendo nosotros los encargados de administrarlos directamente, o que por el contrario se encargue un proveedor. Los más importantes hoy en día son por ejemplo inrupt.net y Solid Community. Este último, ha sido el elegido para la identificación de los usuarios en el proyecto desarrollado.



Figura 4.5 Logo de Solid Inrupt

Para la elaboración del sistema de acceso se han utilizado varias librerías que están disponibles, en este caso: **solid-auth-client**.

Esta librería nos permite acceder a nuestras aplicaciones de una forma segura a través de los pods de Solid, escribir y leer en ellos.

```
//Login
$(function(){
  $('#loginWP').click(function(){
    async function login(idp) {
      const session = await solid.auth.currentSession();
      if (!session){
        await solid.auth.login(idp);
      }
      else{
        // alert('Sesion abierta con ${session.webId}');
      }
    }
    login('https://solid.community');
  })
});

//logout
$(function(){
  $('#logoutWP').click(function(){
    alert("saliendo de solid...");
    solid.auth.logout();
  })
});
```

Figura 4.6 Uso de librería solid-auth

Tal y como se muestra en la imagen se creará una sesión por usuario que será redirigido a la página del proveedor para realizar el logueo. Esta parte de la implementación se ha desarrollado en un archivo Javascript. Este será el archivo maestro en el que se irán ejecutando todas las sentencias necesarias para la elaboración, modificación y borrado de los perfiles de los usuarios.

4.3.1 Carga del perfil

Para llevar a cabo la tarea de la carga del perfil de usuario, se realizan los siguientes pasos. Primero, se comprueba si el usuario ha guardado información en su pod dentro de la carpeta del proyecto, más adelante se creará

en caso de que sea su primera vez. Es allí donde se generará un documento de tipo json, en él se insertarán los datos personales del usuario.

Para tratar y manejar estos metadatos se ha utilizado RDF (Resource Description Framework), ya que es muy útil en circunstancias en las que los datos necesitan ser procesados por aplicaciones que intercambian información. RDF dispone un marco común de trabajo para intercambiar esta información entre aplicaciones distintas mediante "parsers". RDF utiliza URIs para identificar individuos, clases de cosas, sus propiedades y sus valores. Además, dispone de una sintaxis basada en XML para guardar grafos.

Para recoger toda la información que el usuario ingrese en el formulario que le aparecerá al entrar por primera vez en nuestra página se realizará lo siguiente:

```
var FOAF = $rdf.Namespace('http://xmlns.com/foaf/0.1/');
var VCARD = $rdf.Namespace('http://www.w3.org/2006/vcard/ns#');

var profileinfo = {};
profileinfo["name_foaf"] = store.any($rdf.sym(person), FOAF('name')) ? store.any($rdf.sym(person), FOAF('name')).value || null : null;
profileinfo["name"] = store.any(me, VCARD('fn'), null, profile) ? store.any(me, VCARD('fn'), null, profile).value || null : null;
profileinfo["org_name"] = store.any(me, VCARD('organization-name'), null, profile) ? store.any(me, VCARD('organization-name'), null, profile).value || null : null;
profileinfo["org_role"] = store.any(me, VCARD('role'), null, profile) ? store.any(me, VCARD('role'), null, profile).value || null : null;
profileinfo["birthdate"] = store.any(me, VCARD('bday'), null, profile) ? store.any(me, VCARD('bday'), null, profile).value || null : null;

$('#first_login_name').text(profileinfo["name_foaf"]);
$('#first_login > input[name="name"]').val(profileinfo["name"]);
$('#first_login > input[name="country"]').val(profileinfo["country"] || profileinfo["country0"]);
$('#first_login > input[name="region"]').val(profileinfo["region"] || profileinfo["region0"]);
$('#first_login > input[name="city"]').val(profileinfo["city"] || profileinfo["city0"]);
$('#first_login > input[name="postalcode"]').val(profileinfo["postalcode"] || profileinfo["postalcode0"]);
$('#first_login > input[name="address"]').val(profileinfo["address"] || profileinfo["address0"]);
$('#first_login > input[name="email"]').val(profileinfo["mail"] || profileinfo["mail0"]);
$('#first_login > input[name="birthdate"]').val(profileinfo["birthdate"]);
$('#first_login > input[name="org_name"]').val(profileinfo["org_name"]);
$('#first_login > input[name="org_role"]').val(profileinfo["org_role"]);
```

Figura 4.7 Captura de información

De esta manera se ha tratado la información que recibiremos del usuario. Y el formulario del que se nutrirá los datos que ingresemos en el pod del usuario es el que se muestra en la siguiente imagen.


```
do_action( 'woocommerce_before_edit_account_form' ); ?>

<?php do_action( 'woocommerce_edit_account_form_start' ); ?>
<div id="loading" style="display:none;">Cargando datos de Solomon desde tu POD...</div>
</br></br>
<div id="first_login" style="font-weight: bold; display:none; ">
    <!-- Formulario -->
    <!-- Nombre -->
    <div>Hola <a id="first_login_name"></a>. Parece que es tu primer inicio de sesion en
    <br><br>Name: *(Nombre y Apellidos separados por espacios)*<br>
    <input type="text" name="name" value="">
    <br><br>Country:<br>
    <input type="text" name="country" value="">
    <br><br>Region:<br>
    <input type="text" name="region" value="">
    <br><br>City:<br>
    <input type="text" name="city" value="">
    <br><br>Postal code:<br>
    <input type="text" name="postalcode" value="">
    <br><br>Address:<br>
```

Figura 4.8 Formulario de bienvenida

4.3.2 Escritura del pod

Una vez hemos recogido todos los datos, el programa procede a escribirlos en la plataforma de Solid. Para ello, primeramente, crea la carpeta del proyecto donde se va a guardar la información y realiza una petición AJAX al servidor.

De esta manera conseguiremos trabajar de forma asíncrona procesando cualquier solicitud en segundo plano, es decir, de esta forma conseguiremos trasladar la información recibida al lugar de destino.

```
fileClient.createFolder(folder_url).then(success => {
    console.log("Directorio de Solomon creado en: " + folder_url);
}, err => console.log(err) )
.then(sucess => {
    fileClient.updateFile(file_url, contentFile).then( success => {
        console.log("Datos insertados en el fichero de Solomon del POD (" + file_url + ")", contentFile);
        $.ajax({
            type: 'POST',
            data: user_data,
            url: 'shopstore-child/woocommerce/myaccount/aplicacion.php',
            success: function(){ console.log("Ajax enviado "); }
        }).then(success => {
            if(automatic_refresh == true)
                location = location;
        });
    }, err => console.log(err) );
}
);
```

Figura 4.9 Escritura del Pod

El fichero, llamado **aplicación.php** se ha destinado para conectarnos a la base de datos de SQL Server, servicio que nos proporciona la empresa, y si no hay ningún tipo de fallo proceder a la inserccion de los datos del usuario.

```

if( $conn ) {
    print("Conexion abierta con BBDD");
    $data_insert = array();
    $data_insert['NameSurname'] = $_POST['name'];
    $apellidos = explode(" ", $data_insert['NameSurname'] );
    $name = strtok ($data_insert['NameSurname'] , " ");
    $surname = $apellidos[1].$apellidos[2];
    $email = $_POST['email'];
    $gender = $_POST['gender'];
    $birthdate = $_POST['birthdate'];
}

```

Figura 4.10 Escritura en base de datos

```

$sql=
"
    UPDATE [SOLOMON].[dbo].[Users]
    SET     Name='$name',
           Surname='$surname',
           Gender='$gender',
           Birthdate='$birthdate',
           Country='$country',
           Region = '$region',
           City='$city',
           PostalCode='$postalCode',
           Address='$address',
           Languages='$languages',
           TopicsInterest='$topicsInterest'
"

if ( (!sqlsrv_query($conn, $sql, $params)) or (!empty($params)) ) {
    print_r(sqlsrv_errors());
}

```

Figura 4.11 Conexión con SQL Server

4.3.3 Borrado del pod

En este apartado nos vamos a centrar en uno de los puntos fundamentales de toda la recopilación de los datos del usuario, el derecho a la cancelación de los datos recogidos. El usuario tendrá la posibilidad de eliminar todos los datos que haya ingresado en su pod en el momento que lo desee.

Para ello se va a implementar un botón en su perfil de la página. Aparecerá con el nombre “Eliminar datos”. A parte de este botón, dentro de la página Solid Community cuenta con otro botón para su borrado.

```

$('#remove').click(async function loadProfile() {
    // ...
    fileClient.readFolder(folder_url).then(folder => {
        console.log(`Read ${folder.name}, it has ${folder.files.length} files.`);
        // Borrar todos los ficheros de una carpeta
        folder.files.forEach(function(element) {
            fileClient.deleteFile(folder_url + "/" + element.name).then(success => {
                console.log(`Deleted ${file_url}.`);
            });
        });
    });
});

```

Figura 4.12 Borrado del Pod

El código anterior se ejecutará cuando se pulse sobre el botón que hemos habilitado con el id “remove”. De esta manera se borrarán la carpeta dedicada al proyecto, es aquí donde se alojaba la información guardada por el consumidor y que se eliminará.

4.4 Sistema de recomendación

4.4.1 Diseño

Para el desarrollo de nuestro sistema de recomendación se han utilizado tres fuentes distintas de recomendaciones que más tarde se juntarán para obtener el resultado final. Como resumen del trabajo realizado hay que destacar que aun perteneciendo las tres fuentes al sistema de recomendación sólo el primer script es el que realiza la función del recomendador como tal, es aquí donde mediante inteligencia artificial y con el uso de funciones estadísticas se realizan los cálculos, los otros dos scripts se encargarán de segmentar y ajustar los datos devueltos por el recomendador a partir de una serie de interacciones que contaremos en el siguiente punto.

El siguiente diagrama muestra un resumen aclaratorio del diseño creado para nuestro sistema.

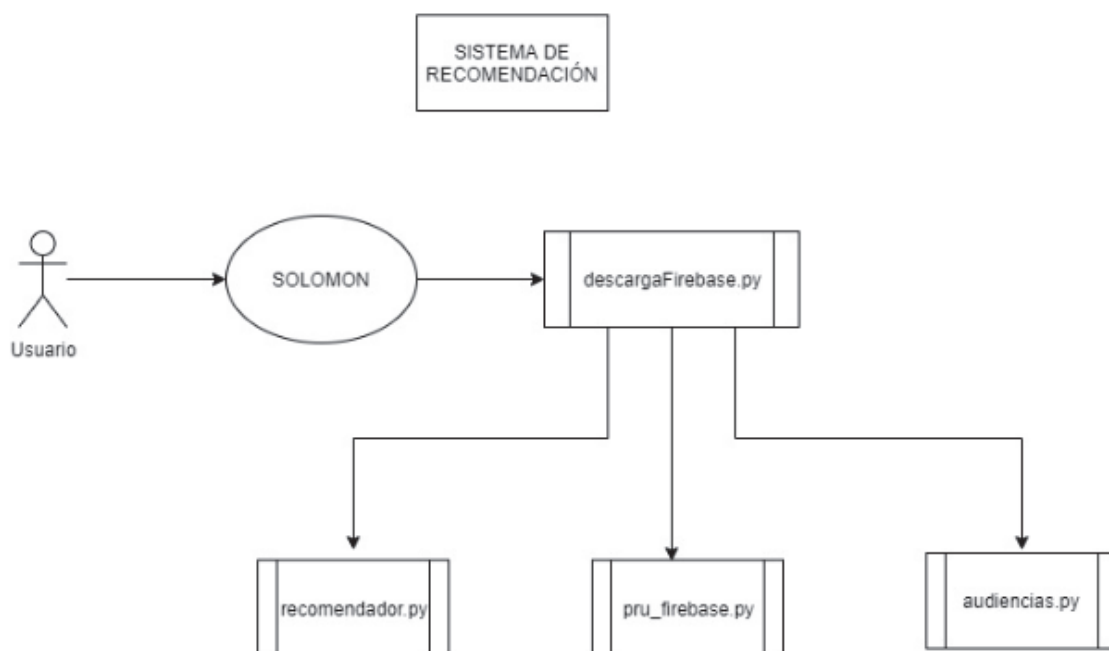


Figura 4.13 Esquema del sistema.

4.4.2 Flujo del sistema

Como hemos comentado en el diagrama del punto anterior, cualquier usuario que entre en nuestra página web comenzará a realizar una serie de acciones como visualizar la tienda online, revisar la sinopsis de un libro que le recomendó un amigo, etc. A medida que se van desarrollando estos hechos, nuestra base de datos se va rellenando de esas interacciones.

En este primer paso entra en juego la herramienta Google Tag Manager (GTM). Como ya hemos mencionado anteriormente este servicio nos va a servir para manejar las distintas actividades que tengan nuestros consumidores.

4.4.2.1 ¿Cómo funciona Google Tag Manager?

Su funcionamiento es muy sencillo. Esta plataforma nos da la posibilidad de crear distintos elementos para controlar la activación de unas etiquetas. Una etiqueta es un fragmento de código en Javascript que se va a enganchar a nuestra página web o a nuestra aplicación. Estas etiquetas van a entrar en juego cuando se den las condiciones establecidas por los activadores que hemos configurado. Y por último, tendremos unas variables que se encargarán de almacenar los datos de seguimiento de la actividad realizada.

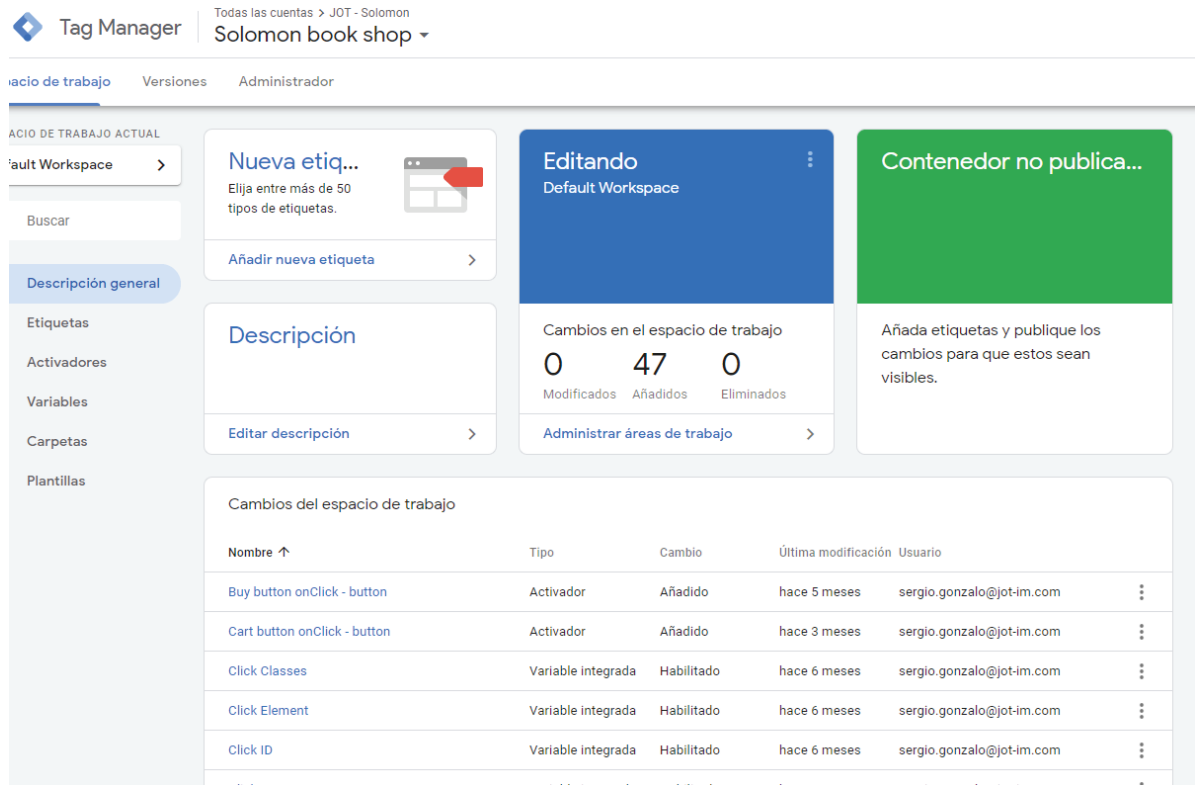


Figura 4.14 Panel principal de GTM

Un ejemplo de toda esta configuración sería el siguiente:

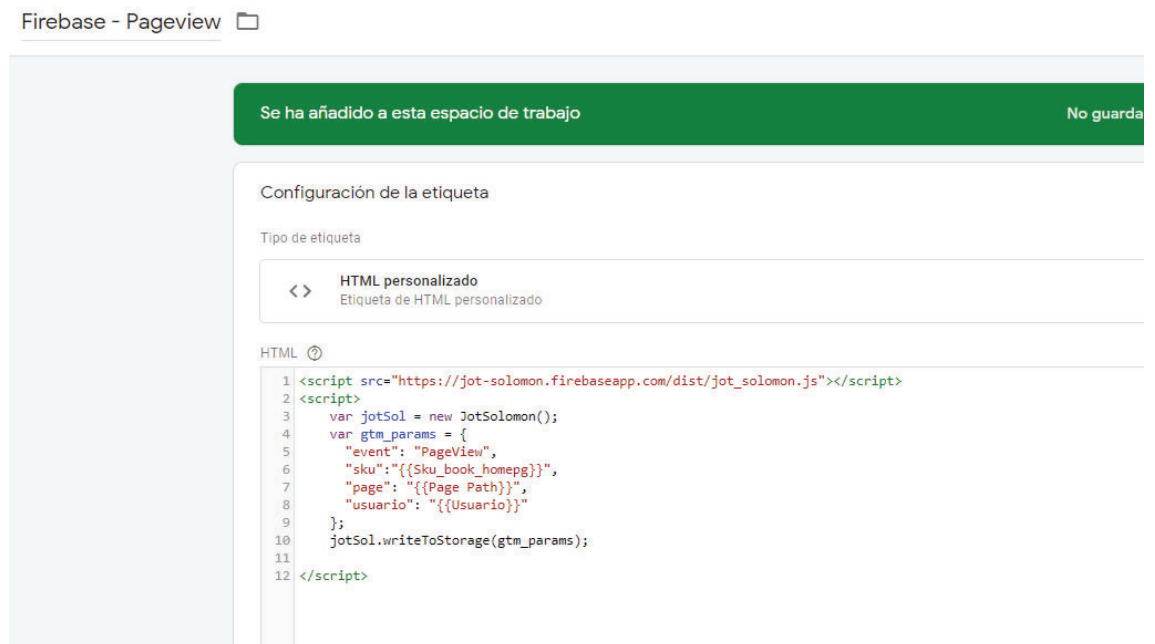


Figura 4.15 Panel de una etiqueta

La captura anterior pertenece a la etiqueta relacionada con la página que se esté visualizando en ese momento. Se obtienen una serie de variables que más adelante serán devueltas en un fichero json y posteriormente tratadas para su análisis por los distintos scripts.

Para finalizar el ejemplo mostraremos la pantalla de configuración del activador.

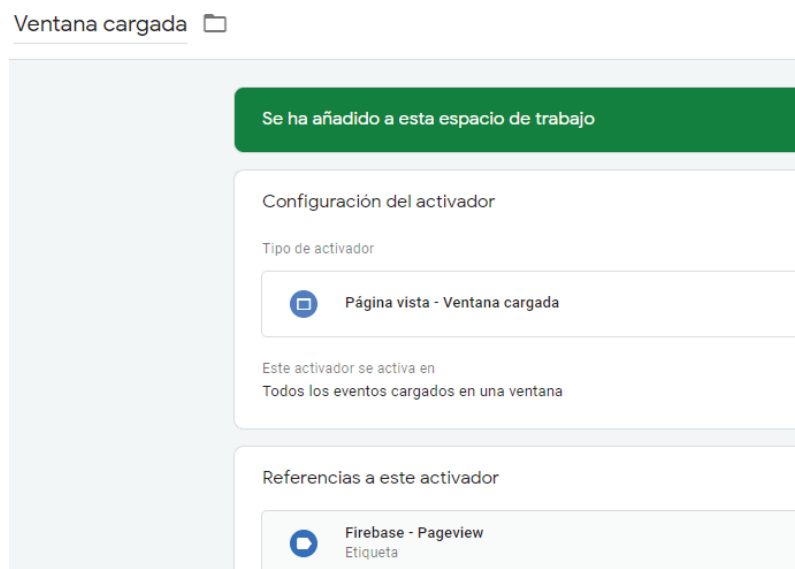


Figura 4.16 Panel de un activador

En esta imagen tan sólo tendremos que relacionar la etiqueta anterior con el activador de ventana cargada que hemos creado. En el momento que se cargue una página se lanzará este activador que pondrá en funcionamiento todo el mecanismo.

4.4.2.2 Descarga de Firebase

Para la realización de este segundo paso se ha desarrollado un script sencillo en Python. Su función principal consistirá en conectarse a nuestro contenedor de Firebase donde hemos ido almacenando la información de GTM. Se leerán los píxeles de seguimiento y se procederá a su descarga.

```
#Inicializacion Firebase
firebaseConfig = {
  "apiKey": "AIzaY2nfjxCRnSRJxti",
  "authDomain": "jot-solomon.firebaseio.com",
  "projectId": "jot-solomon",
  "databaseURL": "https://jot-solomon.firebaseio.com",
  "storageBucket": "jot-solomon.appspot.com"
}
```

Figura 4.17 Datos de configuración para Firebase

Se creará el objeto anterior con la configuración necesaria para poder conectarnos correctamente a nuestro contenedor. Este script contendrá la función descarga que se encargará de, una vez conectados, recorrer los blobs generados y proceder a su descarga en formato json.

```
def descarga():  
  
    firebase = Firebase(firebaseConfig)  
    default_app = firebase_admin.initialize_app()  
    storage_client = storage.Client(project="jot-solomon")  
    buckets = storage_client.list_buckets()  
    bucket = storage_client.get_bucket("jot-solomon.appspot.com")  
    blobs = list(bucket.list_blobs())  
    numero=0  
    print("Descargando blobs ...")  
    for blob in blobs:  
        numero = numero+1  
        pathname = current_dir + "/Blobs/"
```

```
        with open(ruta_bytes, "wb") as json_file:  
            print("descargamos blob")  
            blob.download_to_file(json_file)
```

Figura 4.18 Descarga del blob

4.4.2.3 Script recomendador

Este es el script donde se desarrollará el grueso del sistema. Tanto este, como los siguientes archivos que forman la recomendación se ejecutarán recibiendo como parámetro el email de nuestro usuario conectado. La captura de este parámetro se desarrolla más adelante.

Para llevar a cabo este algoritmo se han utilizado las siguientes librerías. Por supuesto no son las únicas, pero son las más destacables para poder llevar a cabo los cálculos.

```
from scipy.spatial.distance import correlation  
from sklearn.metrics.pairwise import pairwise_distances  
from sklearn.neighbors import NearestNeighbors
```

Figura 4.19 Librerías de scikit-learn

4.4.2.3.1 Librería Scikit-learn

Esta librería es probablemente la más utilizada por los expertos en Data Science para el desarrollo de implementaciones utilizando Machine Learning.

Esta librería nos permitirá utilizar distintos algoritmos de aprendizaje, clasificación, regresión y análisis de grupos en Python.

Dentro de esta librería se utilizarán los módulos **neighbours** y **metrics.pairwise**. El primero se utilizará para el tratamiento de los vecinos cercanos en la matriz de puntuaciones que vamos a crear, en esta matriz se realizará la búsqueda de muestras más cercanas en distancias y predecir el resultado a partir de estas.

El segundo módulo nos ayudará a evaluar distancias en el conjunto de muestras. Las métricas de distancia son funciones $d(a, b)$ tales que $d(a, b) < d(a, c)$ si los objetos a y b se consideran "más similares" que los objetos a y c . Dos objetos exactamente iguales tendrían una distancia de cero.

4.4.2.3.2 Código

Para llevar a cabo el script lo primero que se realizó fue la carga de distintos datasets descargados de internet. Esta información se descargó de la página del Instituto Informático de Fribrugo (IFF) en la cual se proporcionan varios datasets con información de 278.000 usuarios, 271.000 libros y más de un millón de puntuaciones [21]. La empresa JOT, decidió que serían incluidos en la plataforma 749 libros.

La carga de los distintos archivos csv se realizó en dataframes gracias a la librería pandas.

```
#Libros
df_books = pd.read_csv(current_dir + '/datasets/solomonn.csv', sep=';', error_bad_lines=False, encoding='utf-8-sig')
df_books.columns = ['Product_id', 'ISBN_10', 'ISBN_13', 'Title', 'Author', 'Publisher', 'PublishedDate', 'Description',
df_books.drop(['ISBN_10', 'Publisher', 'PublishedDate', 'Description', 'Category_2', 'Price', 'MaturityRating_Google'
```

Figura 4.20 Carga de dataset

Una vez cargados los datasets en distintas variables empezamos a dar forma la matriz de puntuaciones que será la base de los distintos cálculos que hagamos.

Al reducir la empresa el número de libros disponibles fue necesario realizar un cruce de los libros existentes con las puntuaciones de los usuarios. Era necesario que las puntuaciones concordasen con los libros disponibles.

```
#Creacion de nuevo DataFrame con los ISBN y Users que coincidan
ratings_new = df_ratings[df_ratings.ISBN_13.isin(df_books.ISBN_13)]
ratings_new = ratings_new[ratings_new.UserID.isin(df_users.UserID)]
```

Figura 4.21 Cruzado de dataframes

El siguiente paso fue la creación de la matriz de puntuaciones a partir de las puntuaciones de los usuarios a los distintos artículos.


```
ratings_matrix = ratings_new.pivot(index = 'UserID', columns='ISBN_13', values='bookRating')
UserID = ratings_matrix.index
ISBN_13 = ratings_matrix.columns
```

Figura 4.22 Creación de matriz de puntuaciones

A continuación, se ejecutará la función principal. Esta función recibe un id de usuario y la matriz de puntuaciones. Realizaremos la búsqueda de las puntuaciones que no ha realizado el usuario. Una vez completado este paso se procede a realizar las llamadas a las funciones que nos calculan las calificaciones previstas para un libro. Estas funciones son: **predict_itembased** y **predict_userbased**, estos sistemas son los mencionados en el apartado del estado del arte. Corresponden con los sistemas de predicción basados en usuario y basados en el artículo.

```
for i in range(ratings.shape[1]):
    if (ratings[str(ratings.columns[i])][user_id] !=0):
        prediction.append(predict_itembased(user_id, str(ratings.columns[i]), ratings, metric))
    else:
        prediction.append(-1)
for i in range(df_ratings.shape[1]):
    if (df_ratings[str(df_ratings.columns[i])][user_id] !=0):
        prediction.append(predict_userbased(user_id, str(df_ratings.columns[i]), df_ratings, metric))
    else:
        prediction.append(-1)
```

Figura 4.23 Cálculo de las predicciones

Desde esta función también se realiza una llamada a una tercera función. Será la encargada de devolver la correlación de coeficientes entre los usuarios que considere similares, que tienen gustos parecidos.

```
def predict_itembased(user_id, item_id, ratings, metric = metr
    prediction= wtd_sum =0
    user_loc = ratings.index.get_loc(user_id)
    item_loc = ratings.columns.get_loc(item_id)
    similarities, indices=findksimilaritems(item_id, ratings)
    sum_wt = np.sum(similarities)-1
    product=1
```

Figura 4.24 Función para el cálculo basado en artículo

En algunos casos en los que el conjunto de datos era muy escaso y el filtrado de colaboración era casi inexistente devolvía resultados negativos al utilizar la métrica de correlación. Es por ello por lo que se ha asignado el valor uno para resultados negativos.

```
def findksimilaritems(item_id, ratings, metric=metric, k=k):
    similarities=[]
    indices=[]
    ratings=ratings.T
    loc = ratings.index.get_loc(item_id)
    model_knn = NearestNeighbors(metric = metric, algorithm = 'brute')
    model_knn.fit(ratings)
```

Figura 4.25 Función de búsqueda de perfiles similares

4.4.2.4 Script recomendadorFirebase

El objetivo de este segundo script es analizar y procesar la información que hemos descargado desde la plataforma y a partir de esta información calcular nuevas recomendaciones.

Para ello lo primero que se realiza es la carga de los libros que vamos a tener disponibles en la tienda online en un dataframe, tal y como se realizó en el programa anterior.

Se ha creado una función llamada **recorrer_blobs** que se va a encargar de procesar toda la información de los blobs descargados y posteriormente ir insertando cada valor en una variable para ir preparando lo que más tarde será el dataframe encargado de almacenar todas las interacciones de los usuarios (**df_user_interactions**). La siguiente figura sería un ejemplo de la recogida del dato a partir del json descargado:

```
try:
    page = dataframe_leido['gtmParams']['page']
except Exception as exception:
    logging.info('No se encontro page')
    page = ''
```

Figura 4.26 Recogida de datos

Una vez tenemos preparado el dataframe (**df_user_interactions**) ya estaremos preparados para hacer la llamada a la función **dame_recomendaciones**. En ella, se realizará una inserción en base de datos de la información o una actualización del histórico. A continuación, vamos a seleccionar solamente las interacciones del usuario que estamos evaluando y lo depositaremos en otro dataframe, el objetivo es separar la información válida de la que no nos vamos a utilizar.

Llegados a este punto procedemos a evaluar cada interacción del consumidor que está conectado. Se va a puntuar cada acción con una puntuación acorde a su importancia, es decir, no es lo mismo que el usuario A realice una visualización del libro X, a que este mismo usuario A llegue a la página del pago del libro Y, pero en el último momento lo borre del carrito. Se considera que al usuario le ha interesado más el libro Y.

```

if (df1.loc[index, 'EventType'] == 'PageView'):
    df1.loc[index, 'Score'] = 2

elif (df1.loc[index, 'EventType'] == 'Del_cart'):
    df1.loc[index, 'Score'] = 3

```

Figura 4.27 Comparativa de interacciones

Cuando ya hemos realizado todos los cálculos se procederá a evaluar las puntuaciones que ha recibido cada libro a partir del dataset ratings. Con ello lo que queremos conseguir es que en el caso de que no tengamos los datos suficientes para empezar a recomendar se rellenen con los mejor valorados por los usuarios. Por último, se procederá a obtener el score final siendo el valor el valor máximo y cero el mínimo.

```

df_ordenado.loc[i, 'ScoreFinal'] = 1*r['ScoreFinal']/punt_max

```

Figura 4.28: Cálculo final

4.4.2.5 Script recomendador por audiencias

Antes de comenzar este apartado debemos indicar que entendemos o a que nos referimos cuando utilizamos el término “audiencia” o “público”. Conocemos por audiencia o público al conjunto de personas que son usuarios, consumidores o clientes (reales o potenciales) de un servicio, negocio o producto.

Tal y como se explicó en el documento “Descripción de la arquitectura”, en nuestra base de datos se ha desarrollado una serie de tablas donde se relaciona los usuarios con unas determinadas audiencias que explicaremos más adelante.

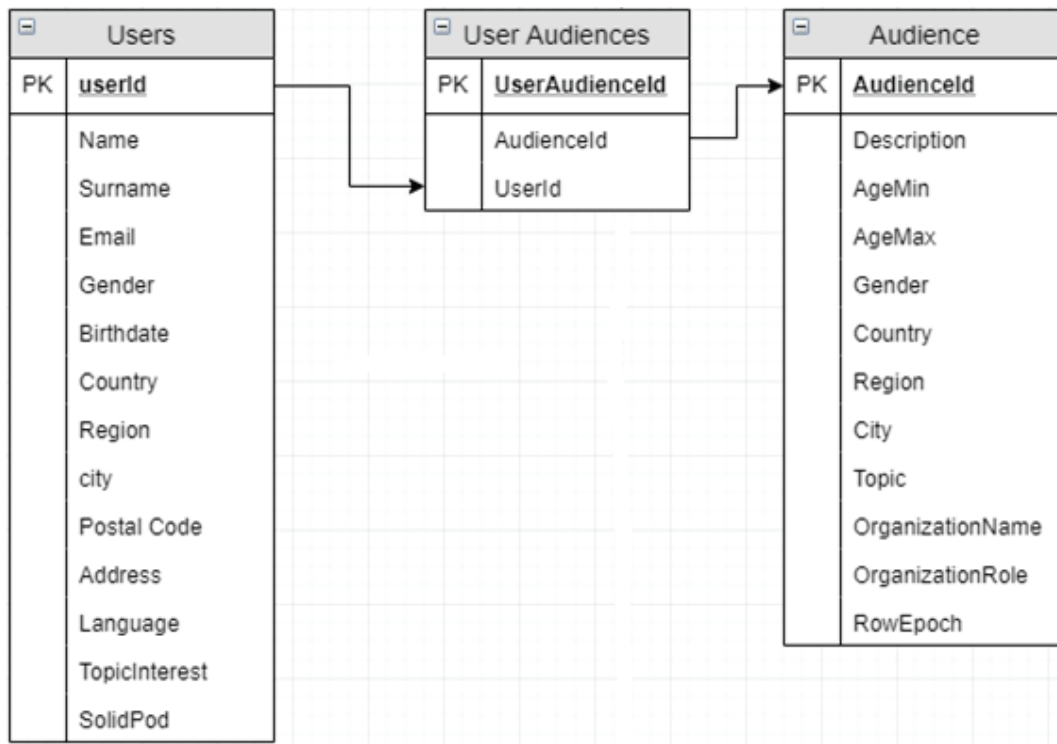


Figura 4.29: Esquema en base de datos

La imagen anterior muestra las distintas tablas que entran en juego a la hora de seleccionar las audiencias correspondientes para cada usuario y como están relacionadas entre sí. El punto de unión será “user_audience”.

La implementación de la parte de audiencias ha sido realizada por la empresa. A pesar de ello, se me explico cómo funcionaría y con qué objetivo. A continuación, se muestra un pequeño ejemplo de la tabla “Audiencias”. La peculiaridad con la que se ha decidido implementar esta tabla ha sido de tal manera que, a partir de una descripción, las columnas contiguas contendrán un valor relacionado o NULL. Es decir, la audiencia “Spanish people” tendrá todos los campos rellenados por el valor NULL o VACIO salvo la columna “Country”, que tendrá el valor “Spain”. De esta manera conseguimos que cualquier usuario que contenga en la columna “País” el mencionado anteriormente, se encontrará dentro de este público. Cada audiencia tiene seleccionado unos libros.

	AudienceId	Description	AgeMin	AgeMax	Gender	Country	Region	City	Topic	OrganizationName	OrganizationRole
14	20	Above 50 travellers	50	NULL	NULL	NULL	NULL	NU...	Travel	NULL	NULL
15	21	Teachers and professors	25	NULL	NULL	NULL	NULL	NU...	Education	NULL	NULL
16	22	Teachers and professors	25	NULL	NULL	NULL	NULL	NU...	History	NULL	NULL
17	23	Teachers and professors	25	NULL	NULL	NULL	NULL	NU...	Science	NULL	NULL
18	24	Teenagers intereseted	12	25	NULL	NULL	NULL	NU...	Humor	NULL	NULL
19	25	Teenagers intereseted	12	25	NULL	NULL	NULL	NU...	Fiction	NULL	NULL
20	26	Teenagers intereseted	12	25	NULL	NULL	NULL	NU...	Comics & Graphic N...	NULL	NULL
21	27	Foodies and singles	25	40	male	NULL	NULL	NU...	Cooking	NULL	NULL

Figura 4.30: Audiencias

5 Costes

En este capítulo vamos a tratar de hacer una aproximación de cuáles serían los costes que ocasionarían la puesta en marcha de este proyecto y los costes relacionados con una futura implementación y mantenimiento. En mi caso he tenido la gran fortuna de que la compañía me ha otorgado el acceso a todas sus tecnologías disponibles, muchas de las cuales, cómo veremos a continuación, disparan su precio a medida que incrementamos su uso. El capítulo se desglosa en dos partes: la primera tratará de las infraestructuras necesarias y la segunda de los costes de desarrollo. Todos los datos que se proporcionan han sido ajustados a la hipótesis de que la herramienta sea usada por unas 1000 personas y durante un año.

5.1 Costes de infraestructura

En este apartado he considerado incluir como imprescindible algunas de las tecnologías que he utilizado para el desarrollo de la herramienta.

5.1.1 Dominio

Este proyecto al tratarse de una página web de recomendaciones de libros lo primero que debemos tener en cuenta es que deberemos comprar un dominio para crear un pequeño atajo práctico y así vincular lo que nuestros consumidores escribirán en la barra de direcciones al entrar en nuestra web. Un ejemplo podría ser *relatbooks.com*. Para esta primera tarea disponemos de una gran cantidad de proveedores que nos ofrecen distintos precios para nuestra propuesta.

Tipo de producto	Coste / mes	Coste total
Dominio	7€	84€

Tabla 5.1: Coste del dominio

5.1.2 Material

A continuación, se muestra una tabla con unos datos de ejemplo asociados al material necesario para poner en marcha el proyecto.

Material	Cantidad	Precio/u	Total
Portátil HP Pavilion 14-ce3003ns	1	799€	799€
Monitor HP 24fw	2	158€	316€
Ratón inalámbrico HP Z3700	1	20€	20€
Base de acoplamiento HP	1	168€	168€
Cable HDMI	2	12€	24€
			1327€

Tabla 5.2: Coste del material

5.2 Costes de desarrollo

En este segundo apartado referido a los costes se han insertado aquellos que son necesarios para la creación de la herramienta.

5.2.1 Licencias software

Nuestra plataforma manejará y albergará una gran cantidad de datos de los usuarios, por tanto, es imprescindible tener toda esta información estructurada y almacenada. Para ello se ha decidido utilizar el servidor de bases de datos SQL Server.

Por otro lado, para la escritura y almacenamiento de toda la información que Google Tag Manager recoge (acciones de los consumidores) la controlaremos con Firebase Google Cloud. Esta plataforma permite a las aplicaciones pequeñas que no superen las 100.000 escrituras y eliminaciones, como sería nuestro caso, un coste mensual de 11 dólares.

Licencia/Producto	Tipo	Coste individual	Coste total
SQL Server 2019	Servidor Cal	+ 188€ *	188€*
Firebase Google Cloud		9.91€/mes	119€

Tabla 5.3: Coste de las licencias

*Sujeto a volumen

5.2.2 Desarrollador

El proyecto realizado se enmarca en el plan establecido de 12 créditos ECTS que requiere un esfuerzo total planificado de 324 horas de trabajo (27 horas/ECTS * 12 ECTS).

A continuación, se muestra el listado de tareas realizadas para la elaboración y documentación de la herramienta que se ha realizado a lo largo de 81 días de trabajo. Además, se inserta también el diagrama de Gantt referente al mismo donde se muestran desfases del tiempo previsto y el real.

Actividad 01	Investigación y estudio del estado del arte.
Actividad 02	Diseño de la arquitectura del Sistema.
Actividad 03	Diseño de la tienda online.
Actividad 04	Creación del Wordpress.
Actividad 05	Ajuste de las plantillas, temas y elementos del Wordpress.
Actividad 06	Desarrollo del <i>Child Theme</i> de Wordpress.
Actividad 07	Descarga de los productos a introducir en la tienda.
Actividad 08	Carga de estos productos al Wordpress.
Actividad 09	Diseño de la base de datos.
Actividad 10	Diagrama Base de Datos.
Actividad 11	Desarrollo de la arquitectura de la Base de Datos.
Actividad 12	Inserción de los datos necesarios en la base de datos.
Actividad 13	Diseño del módulo de trackeo y medición.
Actividad 14	Creación del espacio en Google Tag Manager.
Actividad 15	Diseño de los pixeles de Google Tag Manager.
Actividad 16	Creación de los activadores y etiqueta de Google Tag Manager.
Actividad 17	Desarrollo de las variables de Google Tag Manager.
Actividad 18	Guardar interacciones destacadas (pixeles) del usuario en Google Tag Manager.
Actividad 19	Creación del espacio en Solid Community (Login Usuarios).
Actividad 20	Guardar datos del usuario en la página de Solid Community (Login).
Actividad 21	Guardar datos del formulario de bienvenida en base de datos (Usuarios).
Actividad 22	Creación del espacio de almacenamiento en Firebase.
Actividad 23	Ajuste en código para almacenamiento en Firebase.
Actividad 24	Enlazar Firebase y Google Tag Manager.
Actividad 25	Diseño del motor de recomendación.
Actividad 26	Desarrollo del motor de recomendación.
Actividad 27	Ajuste del sistema recomendación al Wordpress.
Actividad 28	Elaboración de la memoria del proyecto.
Actividad 29	Elaboración de la presentación.

Figura 5.1: Lista de tareas realizadas

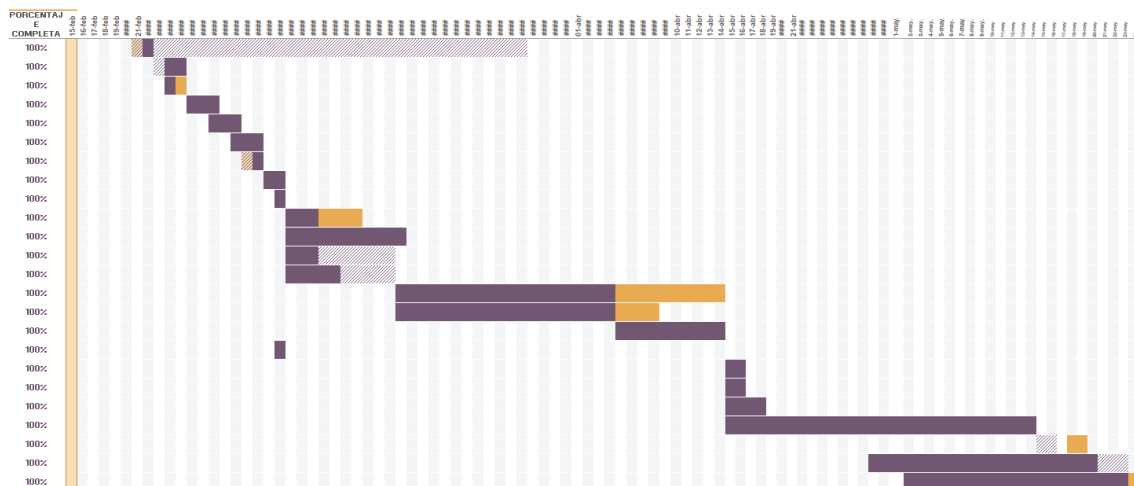


Figura 5.2: Diagrama de Gantt

Teniendo estos datos en cuenta, se ha procedido a la elaboración de los costes que tendría, en nuestro caso, un programador Junior. En términos estadísticos, la media de este tipo de desarrolladores en España asciende a unos 20.000€ al año. Por tanto, calculamos que el salario diario sería de unos 55 € aproximadamente. Unos 6,5€ la hora trabajada haría un total de 2.106€.

Trabajador	Precio/hora	Horas trabajadas	Total
Desarrollador Junior	6,5€	324h	2106€

Tabla 5.4: Coste del desarrollador

5.3 Coste final

Una vez hemos realizado todas las cuentas aproximadas del proyecto obtenemos un total de 3824€.

6 Resultados

A lo largo de este capítulo se van a mostrar las diferentes pantallas que han resultado de la elaboración del proyecto. Además, se explicará de forma detallada cuales son los pasos necesarios para poder disfrutar al máximo de la herramienta.

6.1 Acceso

En primer lugar, para acceder a nuestra cuenta en la página de Solomon deberemos hacer click en el link de “Mi cuenta”, que nos llevará a la pantalla que se muestra a continuación. Si pulsamos en el botón “Login”, nos va a redirigir a la página Solid-Community, donde deberemos acceder con nuestro Pod, en el caso de tenerlo, o en caso contrario procederemos a crearlo con nuestros datos. Más adelante explicaremos mediante imágenes ambas situaciones.

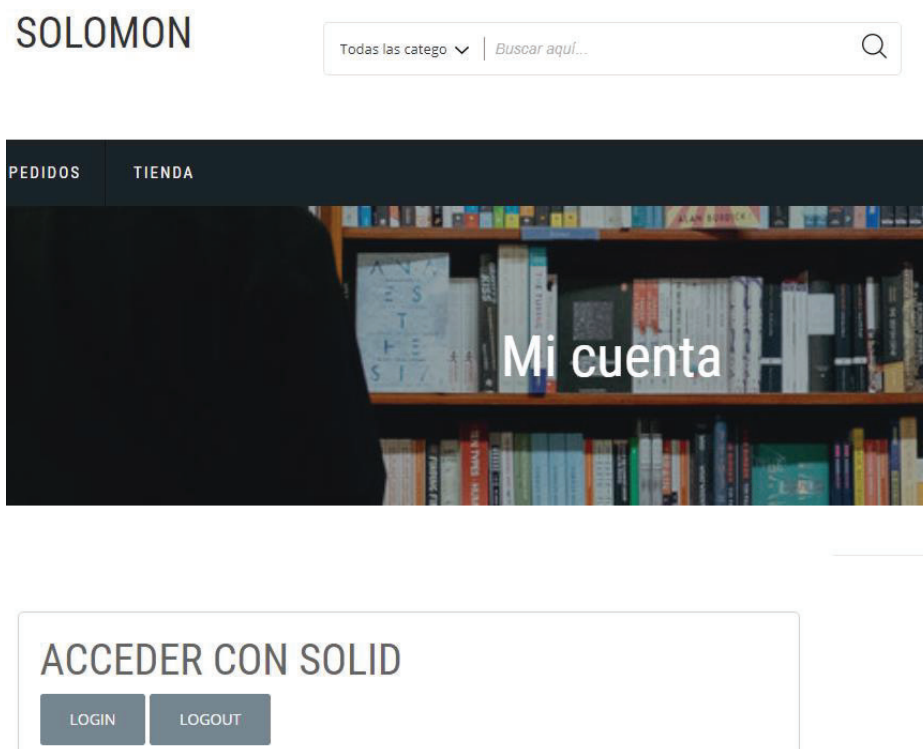


Figura 6.1: Acceso a Solomon

La siguiente imagen muestra la pantalla de acceso a Solid Community. En este paso, podremos o acceder con nuestros datos o crear una cuenta en caso de ser no tenerla.

Login

Username

Password

Log In

Forgot password?

Log in with Certificate

New to Solid? [Create an account](#)

Figura 6.2: Login Solid

Si no tenemos creado nuestro Pod tendremos oportunidad de hacer si pulsamos en el link “Create an account” que aparece debajo del botón “Login”. El link anterior nos llevará a una página donde tendremos que rellenar un formulario donde introduciremos algunos datos que se guardarán en nuestro Pod.

6.2 Registro

Register

Username*

alice

Your username should be a lower-case word with only letters a-z and numbers 0-9 and without periods.

Your public Solid POD URL will be: `https://alice.solid.community`

Your public Solid WebID will be:
`https://alice.solid.community/profile/card#me`

Your *POD URL* is like the homepage for your Solid pod. By default, it is readable by the public, but you can always change that if you like by changing the access control.

Your *Solid WebID* is your globally unique name that you can use to identify and authenticate yourself with other PODs across the world.

Password***Repeat password*****Name*****Email***

Your email will only be used for account recovery

☐ Connect to External WebID (**Advanced feature**)

Register

Figura 6.3: Registro del Pod

Si hemos introducido los datos correctamente y la contraseña cumple los requisitos establecidos por seguridad, la pantalla que aparecerá será la página principal de Solid con nuestro Pod creado.

6.3 Pod

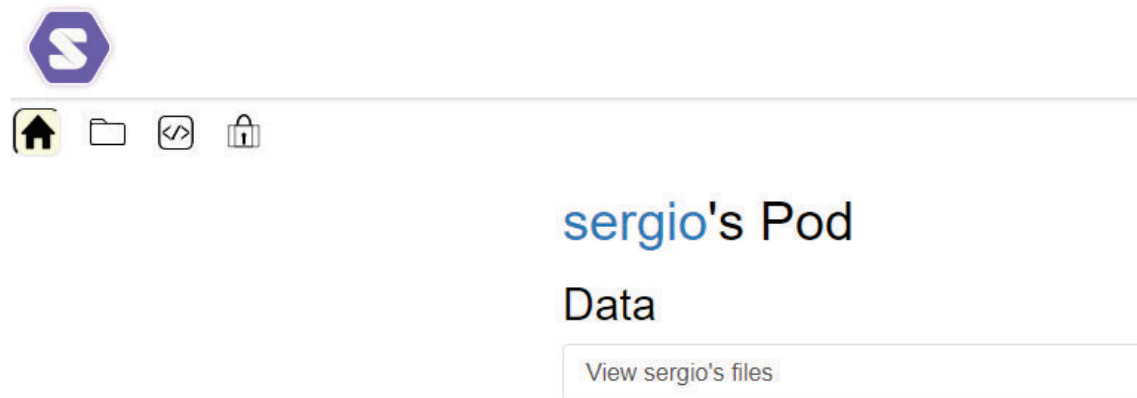


Figura 6.4: Panel del pod

Una vez realizado este paso ya estaremos listos para acceder a la página de Solomon con nuestra cuenta. En el caso de que no hayamos rellenado de datos nuestro Pod hará que el mensaje que aparezca después de acceder sea el siguiente:



PEDIDOS

TIENDA

Mi cuenta

[› Pedidos](#)[› Descargas](#)[› Direcciones](#)[› Detalles de la cuenta](#)[› Salir WP-SS](#)

Hola sergio. Parece que es tu primer inicio de sesion. Por favor, comprueba tu informacion del POD y rellena lo que falte:

Nombre ***Pais ***

Figura 6.5: Formulario de registro

Esta página expone un pequeño formulario con información básica que almacenaremos en nuestro Pod. En él se encuentra un apartado donde ingresaremos cuales son los nuestros gustos, más adelante estos tomarán importancia.

Se han encontrado datos de Solomon en tu POD:

org_name	JOT
org_role	Dev-
topics_interest	Computers
languages	es
solidPod	https://sergiogonzalo.solid.community
registerDate	2020-2-24 10:17:14
currentSession	2020-2-24 10:17:20
lastSession	2020-2-24 10:17:14

Eliminar datos Solomon

Figura 6.6: Panel de los datos insertados

Una vez introducidos los datos en el formulario anterior y tras haber sido guardados nos aparecerá un aviso para recargar la página. Si hacemos lo que nos indica, volverá a cargar la página, pero esta vez nos mostrará en la pantalla los datos que hemos introducido. Además, el usuario tendrá la opción en cualquier momento de eliminar los datos que ha introducido pulsando el botón que aparece abajo, “Eliminar datos”.

6.4 Tienda

En cualquier momento de nuestra etapa en la página de Solomon podremos acceder a la tienda, ya sea con los datos del Pod rellenados o sin ellos, pero la experiencia del sistema de recomendación será distinta ya que aún no tendremos información acerca de los gustos que se recogen en el formulario del principio.

La pantalla que se muestra a continuación es un ejemplo de lo que se mostrará al acceder a la tienda, donde se mostrarán algunos libros disponibles.

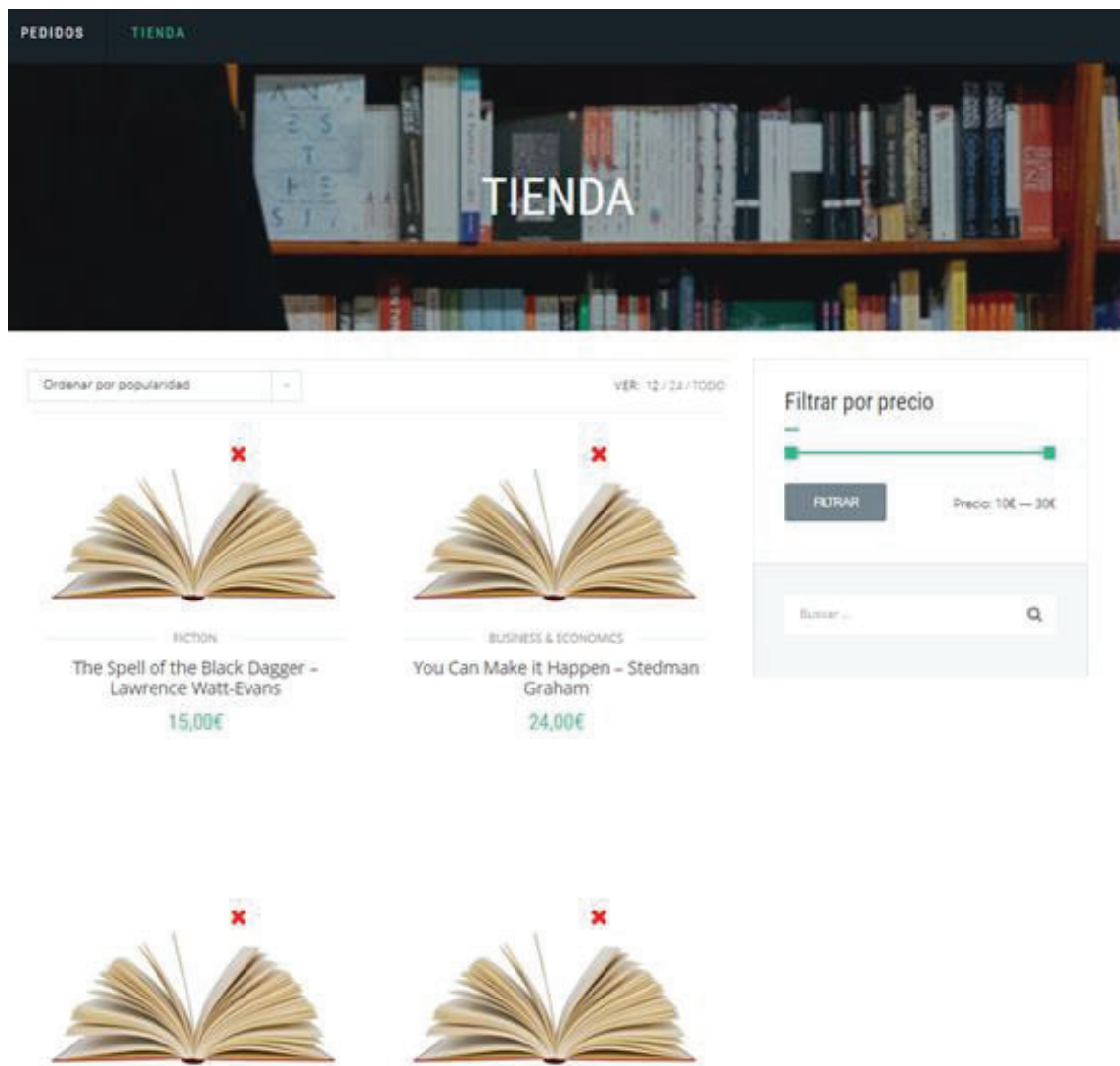


Figura 6.7: Página principal de la tienda

6.5 Recomendaciones

Si alguno de los libros es del agrado del consumidor y decide pulsar sobre alguno de ellos, esta acción le llevará a la página principal del producto. Aquí, aparecerá debajo del título un apartado para calificar cual ha sido su experiencia con ese ejemplar, esto servirá a otros usuarios para saber la valoración que le han dado otros usuarios. Además, recogerá en caso de ser rellenado la puntuación que le otorgue el cliente y que será almacenado por el sistema de recomendación.

Debajo del libro aparecerá un cuadro con cuatro recomendaciones basadas en el perfil de cada usuario.

6.5.1 Ejemplo de uso

En este apartado se va a realizar una pequeña demostración del proceso final que tendría la aplicación web y como lo vería el cliente.

Supongamos que accedemos a la página de Solomon y queremos comprar un regalo a un familiar. Sabemos que a nuestro familiar le encanta la cocina y todo lo relacionado con la elaboración de platos caseros. Así que empezamos a buscar ejemplares como el que se muestra:

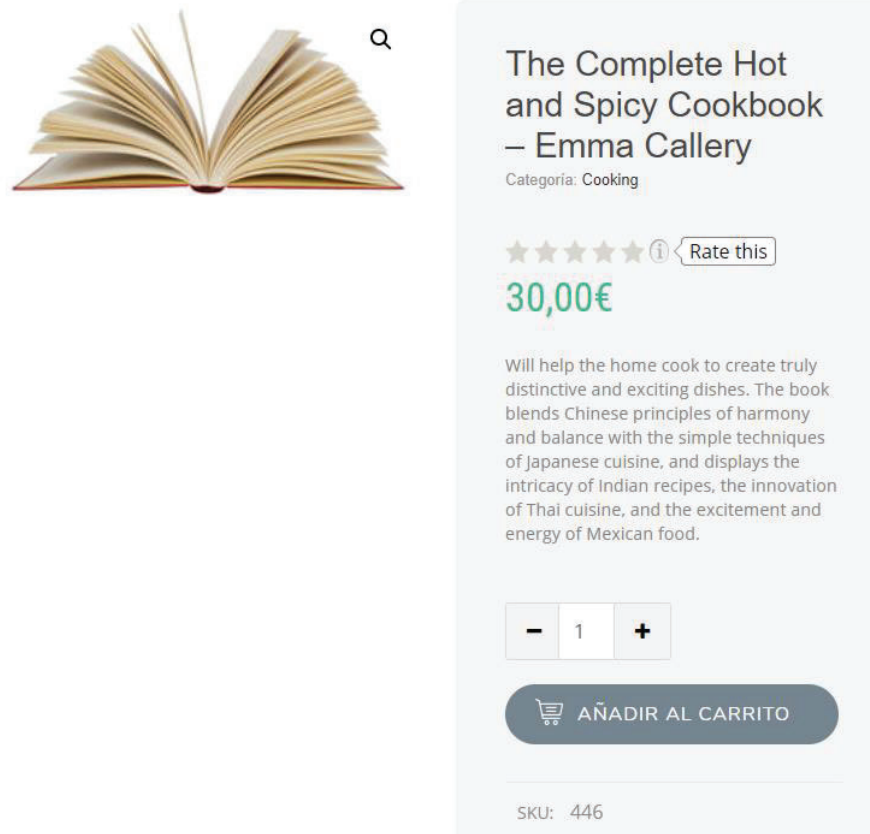


Figura 6.8: Página principal de un artículo

Por el momento nos convence y decidimos añadirlo al carrito. Pero en el último momento nos arrepentimos y decidimos esperar a ver si hay alguno que nos convence más.

		Producto	Precio	Cantidad	Subtotal
		The Complete Hot and Spicy Cookbook - Emma Callery	30,00€	- 1 +	30,00€

Subtotal	30,00€
Total	30,00€

Figura 6.9: Menú del carrito

En la parte final de la página empezamos a comprobar una serie de artículos relacionados. Estos irán acercándose a nuestros gustos a medida que vayamos realizando interacciones en la plataforma. El resultado sería como se muestra en la siguiente imagen.

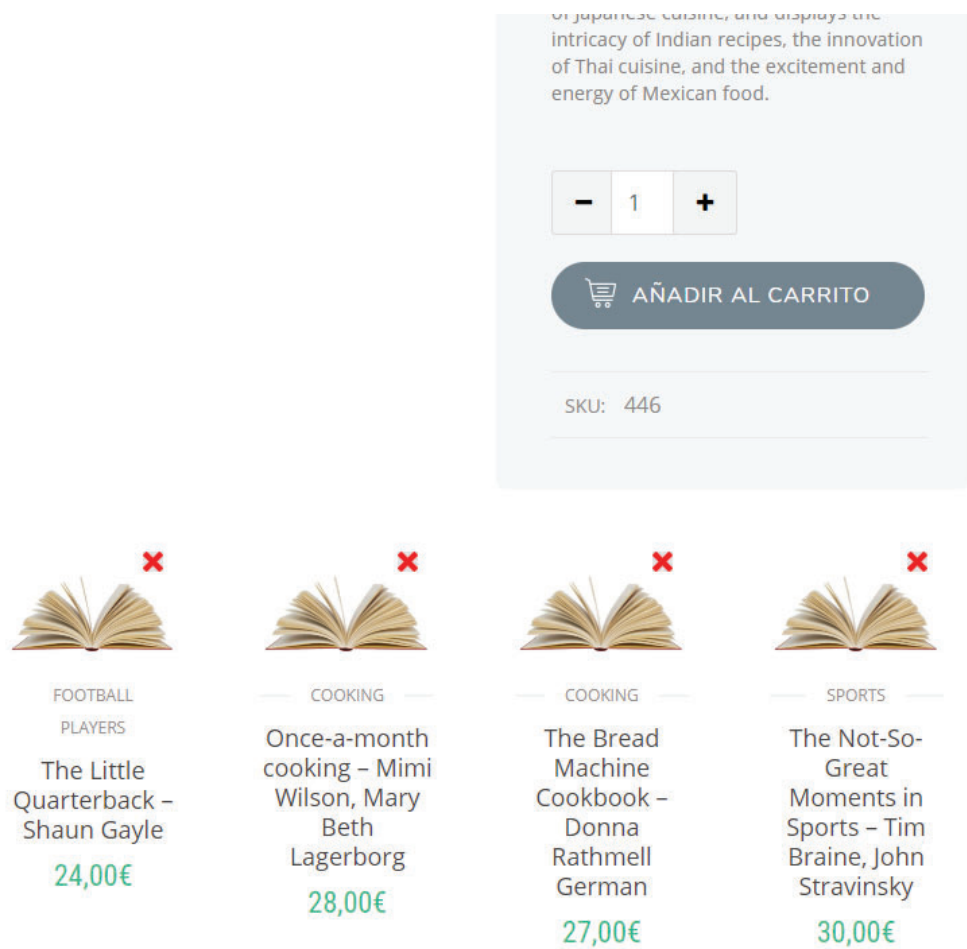


Figura 6.10: Recomendaciones mostradas

Como se puede comprobar aparecen una serie de recomendaciones, pero dado que unos días atrás realizamos unas búsquedas del nuevo libro de un periodista deportivo hemos obtenido también algunas recomendaciones diferentes.

7 Conclusiones

Tras el desarrollo de este proyecto, se puede concluir que se ha conseguido realizar los objetivos propuestos al principio de su elaboración a pesar de no haberse cumplido con los plazos previstos en el diagrama de Gantt, debido a diversas dificultades acontecidas. Como consecuencia, se ha conseguido crear un sistema de recomendación híbrido basado en una tienda de libros, donde cualquier usuario que la utilice empezará a obtener recomendaciones a partir de su actividad.

Gracias a la realización de este proyecto he conseguido desarrollar mis conocimientos en las herramientas que se han usado a lo largo del trabajo como SQL Server y Google Tag Manager, herramientas que seguro me sirven para mi futuro profesional. La primera de ellas, aunque la conocía y la había usado, haciendo un balance ahora, me doy cuenta de que desconocía la gran mayoría de características debido al mínimo uso que le había dado y GTM desconocida totalmente. Y por supuesto, del aprendizaje de nuevos lenguajes de programación que, durante mi etapa en la universidad, debido a las optativas elegidas, desconocía por completo. Creo que este ha sido uno de los puntos más difíciles que he tenido que afrontar, aprender de forma acelerada un lenguaje desconocido y de forma autónoma. Pero ahora echando la vista atrás, valoro mucho más tras haber superado esta piedra del camino.

8 Bibliografía

Entre paréntesis se indica la fecha de la última consulta

- [1] Perfil de usuario, (21/5/2020), https://es.wikipedia.org/wiki/Sistema_de_recomendaci%C3%B3n
- [2] JOT Internet Media, (28/5/2020), <https://www.jot-im.com>
- [3] Filtrado colaborativo, (21/05/2020), https://es.wikipedia.org/wiki/Filtrado_colaborativo
- [4] Método del vecindario, (21/05/2020), <https://towardsdatascience.com/intro-to-recommender-system-collaborative-filtering-64a238194a26>
- [5] Distribución de Pearson, (22/05/2020), https://es.wikipedia.org/wiki/Distribuciones_de_Pearson
- [6] Índice de Jaccard, (22/05/2020), https://en.wikipedia.org/wiki/Jaccard_index
- [7] Sistemas de recomendación basados en contenido, (23/05/2020), https://en.wikipedia.org/wiki/Recommender_system#Content-based_filtering
- [8] Python, (28/05/2020), <https://es.wikipedia.org/wiki/Python>
- [9] Librería Pandas, (23/05/2020), <https://www.learnpython.org/es/Pandas%20Basics>
- [10] Lenguaje Java, (23/05/2020), <https://desarrolloweb.com/articulos/497.php>
- [11] Lenguaje Javascript, (23/05/2020), <https://es.wikipedia.org/wiki/JavaScript>
- [12] PHP, (23/05/2020), <https://desarrolloweb.com/home/php>
- [13] Lenguaje PHP, (23/05/2020), <https://es.wikipedia.org/wiki/JavaScript>
- [14] Microsoft SQL Server, (23/05/2020), https://es.wikipedia.org/wiki/Microsoft_SQL_Server
- [15] SQL Server, (28/05/2020), <https://openwebinars.net/blog/que-es-sql-server/>
- [16] Firebase, (28/05/2020), <https://es.wikipedia.org/wiki/Firebase>
- [17] Firebase , (28/05/2020), <https://rockcontent.com/es/blog/que-es-firebase/>
- [18] Google Tag Manager, (28/05/2020), <https://seocom.agency/es/blog/google-tag-manager-que-es-como-funciona/>
- [19] Wordpress, (28/05/2020), <https://es.wikipedia.org/wiki/WordPress>
- [20] Solid Pods, (28/05/2020), [https://es.wikipedia.org/wiki/Solid_\(proyecto_de_descentralizaci%C3%B3n_web\)](https://es.wikipedia.org/wiki/Solid_(proyecto_de_descentralizaci%C3%B3n_web))
- [21] Datasets, (28/05/2020), <http://www2.informatik.uni-freiburg.de/~chiegler/BX/>

9 Anexo

Se adjunta el documento de la empresa que permite su publicación.

La empresa JOT Internet Media CIF: B84135144 autoriza que el estudiante Sergio Gonzalo Vallinas, con DNI 47298147-G, pueda utilizar la información a la que haya podido acceder durante el desarrollo de las actividades en la empresa, para la elaboración de la memoria del trabajo de fin de grado, así como su presentación y, en su caso, su defensa pública.

Para que conste a los efectos oportunos firmo el presente documento

Madrid a 2 de Junio de 2020



Fdo: Jose Cabanillas Alcázar
DNI: 50810436-D
Director técnico de JOT Internet Media S.L.

Este documento esta firmado por



Firmante	CN=tfgm.fi.upm.es, OU=CCFI, O=Facultad de Informatica - UPM, C=ES
Fecha/Hora	Fri Jun 05 16:34:37 CEST 2020
Emisor del Certificado	EMAILADDRESS=camanager@fi.upm.es, CN=CA Facultad de Informatica, O=Facultad de Informatica - UPM, C=ES
Numero de Serie	630
Metodo	urn:adobe.com:Adobe.PPKLite:adbe.pkcs7.sha1 (Adobe Signature)